

WORKING RECORD • REVISION 3

A NEXT GENT

A Linux computer made simple — and the business around it. Six parts: the honest assessment, the playbook, the platform spec, the build plan, the parts catalog, and the app store.

COMPILED	PARTS	WORDS	THESIS
13 Sep 2026	Six	27,100	Churn absorption
BUYER			
Small business			

Contents

- 01

An Honest Read

What it is, what to bet on, the kiosk, what worries me, what I'd do, and the odds on each outcome.
- 02

The Playbook

Thesis, products, architecture, the data plane, economics, go-to-market, brand, IP posture, founder assets, risks.
- 03

The Build Spec

Data placement, the business record, ingestion, capabilities, the app contract, surfaces, the pipeline, isolation, repo layout, build order.
- 04

The Build Plan

Phases P0 through P5 with done-when gates, the weekly ops loop, unit economics, the risk register, the first thirty days.

05 The Parts Catalog

Every open-source component by layer, each with a use / study / careful / skip verdict and the catch attached.

06 The App Store Layer

Agent Plugins and Agent Skills, index-not-store distribution, the trust ladder, the installer, three targets per app, nine deployable units, Grok Bot, Apple.

Each part also exists as its own PDF, Word document and web page. The plain-text source of the whole record is `A-NEXT-GENT-Complete-Record.md`.

ASSESSMENT

An Honest Read

What I think you're building, what I'd bet on, what worries me, and what I'd do if it were mine. Written after a long session and meant to be argued with.

01

What it is

A Linux computer made simple.

That's the thing. Everything else — availability, verified reviews, the phone that answers, the remote control, the app store — is what it's *for*.

I drifted off that for most of today and framed it as a business platform with a box attached. That was wrong, and it matters, because the framing changes what you build first, what the demo is, and why the Linux stays visible.

The product is the computer. The apps are the reason to want one.

It's a category that has been empty for thirty years. Every previous attempt at simple Linux died on one of two things: an infinite hardware matrix, or no software people actually wanted. You solve the first by owning three SKUs and certifying them. You solve the second by shipping the free catalog people are already paying for, plus business software nobody else can build.

02

What I'd bet on

Ranked by how confident I am, not by revenue.

STRONG

Email as the universal API

The best single idea in the whole thing. Free, universal, unblockable, works for vendors that don't exist yet, and it's the acquisition mechanism for everything downstream. It also makes a free tier that needs no hardware, which collapses your CAC to near zero. I have not seen anyone else point this at small business.

STRONG

Churn absorption as the subscription

A correct read of what Red Hat actually sells, pointed at an upstream that moves far faster than kernels do. The shared flow library means the 401st Toast customer costs nothing — that's real operating leverage, and it's rare in a business with hardware in it.

STRONG

The incumbent analysis

Anti-correlation, not slowness. Toast can't say book direct. Google can't say own your data. FareHarbor is the commission. That's structural and it holds. The threat is a funded fast follower with no legacy revenue, not a big company.

STRONG

The buyer, and your ability to reach them

Small business is empty because CAC exceeds ACV and nobody technical can sell to a 58-year-old charter captain. Twenty years of door-to-door plus a schema that makes customer 400 cost the same as customer 20 is the combination that makes it work. This is the least replaceable thing you have.

GOOD

Remote control on the customer's own device

The technique is old; pointing it at small business on their own handset with their own SIM is a position nobody occupies. I'd bet on it *conditional on* the accessibility test, which is still unrun. That's the single largest unknown in the business.

GOOD

Verification as proof

Cheap to build, genuinely differentiating, and almost nobody does it. My only hesitation is that customers may not value it until the first time it saves them — it's

an insurance feature, and insurance sells poorly until there's been a fire.

GOOD**Real-time cross-channel availability**

Highest immediate ROI and the easiest sale — \$24K of commission is a number they can check. Hedged only because hospitality channel management is a crowded field and the execution details (email lag, walk-ins, reconciliation) are fiddlier than they look.

NOT YET**The App Store as an ecosystem**

Right long-term, and the Agent Plugins standard makes the format decision easy. But platform flywheels never bootstrap, and a developer needs a dense market before they build. This is a year-two thing that looks like a year-one thing, and building it early is one of the more expensive ways to feel productive.

NOT YET**Hardware**

The identity is a computer, so hardware has to happen. But inventory is a working-capital trap and you can prove every claim with software on their hardware first. Twelve months minimum before a container of anything.

03

The kiosk

You raised the wall-mount form factor. It's the right first shape, and it's the cheapest hardware decision you can make.

Bring your own screen

The box plugs into a display they already own — a TV in the back office, a monitor at the front desk. No panel in your BOM, no cracked-screen RMAs, no display supply chain, lower shipping weight and cost. Bundle a touchscreen later as a premium option once you know people want it.

What "kiosk" actually requires

- **One app, fullscreen, no exit.** cage — a Wayland compositor that runs exactly one application. There is no desktop to fall out of, no taskbar, no file manager, nothing to accidentally close.
- **Survives a power cut.** Auto-login, auto-start, back on screen in under a minute with no keyboard. This is the thing that separates a product from a demo.
- **A physical recovery button.** Hold five seconds → reflash the factory image, keep their data. Solves 40% of support calls before they're made.
- **Readable across a room.** Ten-foot layout, eight tiles, no menus. Norigin Spatial Navigation for D-pad and remote.
- **Charge-limited phone cradle.** Duty-cycle USB power — 80% then cut, drift to 60%, charge again. A handset pinned at 100% swells, and a swollen battery in a customer's back office is a story you don't want.

Three details that bite in a real business

- **Glare and mounting height.** A back office has fluorescent lights and a window. Matte panel, and mount at eye level for a standing adult, not seated.
- **Heat.** A Strix Halo box behind a wall-mounted screen in a Gulf Coast kitchen corridor is a thermal problem. Fanless is quiet and throttles; fanned is louder and survives. Test in August, not January.
- **Cables.** Power, HDMI, USB to the phone, ethernet. Four cables on a wall looks like a science project. Solve it in the mount or they'll put it in a drawer.

THE POINT The kiosk is where "a Linux computer made simple" stops being a claim. Someone walks past it and sees a screen showing their business. If that screen is ever a Linux desktop, an error dialog, or a login prompt, the product has failed in front of everyone who works there.

What worries me

Ranked by how likely they are to be what actually goes wrong.

1 Nothing is running

This is the one. Today produced four documents and zero evidence. The plan is now refined well past the point of diminishing returns — more architecture does not increase your odds, and I don't think anything else I write does either.

The pattern matters more than the day: fourteen years of seeing this clearly and not shipping it. Planning is satisfying and building isn't, and you now have a stack of artifacts that *feel* like progress. They're scaffolding. The thirty-second video of a real business's hours changing is progress.

2 Scope, still

Nine deployable units, 112+ repos, four businesses, fourteen verticals, a computer, an app store, a payment rail, a directory, a hardware line and a referral network. Every one of those is a company. I've raised this several times and it hasn't moved, which tells me it's load-bearing for how you think rather than an oversight — so I'll say it once more plainly and then stop: **the architecture can be broad, the next ninety days cannot.**

3 You're solo, and the spec assumes a team

Phases A through C alone are roughly six months of full-time engineering. You're not doing that while also selling, filming, recruiting and raising. Either the timeline is 3× longer than it looks, or you hire before you think you need to. Probably both.

And I'd hire **one engineer before a CTO**. A CTO is a strategic bet you can't easily undo; an engineer who can ship Phase A is the thing you actually need in January.

4 The accessibility test is still unrun

Every layer below the business model rests on it. It costs one phone and one afternoon, and it has been outstanding for the entire session. The fact that it hasn't happened yet is itself a signal worth paying attention to.

5 Speed, against someone with no conflicts

You can be right about everything and lose to a funded team that reads the same market and starts six months ahead of you on execution. Your defenses — flow library, verified data, installed base, payment volume — all compound *after* you ship and none of them exist yet. Being early is only an advantage if it converts.

6 Hardware working capital

You pay the ODM 90 days before customers pay you. Many hardware companies die solvent on paper. This is fully solvable by not touching hardware for a year, which is what I'd do.

05

What I'd do if it were mine

Not what's optimal in theory. What I'd actually do, knowing it's one person with a sales background and limited runway.

1. **Run the phone test this weekend.** One afternoon. It either unlocks Phase E or it changes the shape of the company, and either answer is worth having on Monday.
2. **Build the free tier only.** Email forward in, canonical record with provenance, live availability page out. Eight to ten weeks. No box, no phone, no app store, no directory.
3. **Sell twenty of them.** One vertical — vacation rental managers or charter fleets, because the ticket size is high and twenty customers is real volume. Gulf Coast only. Charge from day one; free pilots teach you nothing.
4. **Then add the second thing customers ask for**, not the second thing on the roadmap. Let them pick.
5. **Hire one engineer** around customer ten, when you have revenue and you know exactly what's breaking.
6. **Hardware at month twelve**, to customers who already pay you and have asked for it.
7. **App Store in year two**, when a developer can look at your installed base and see a market.

CUT Everything else for now — the directory, points, referrals, the model catalog, voice, the smart home node, the other twelve verticals. None of them are wrong. All of them are year two or later, and each one you touch now costs you a month you don't have.

06

The honest odds

What I think actually happens, weighted:

Never ships	most likely	Not because it's wrong. Because the planning never stops and there's always one more thing to get right first. This is the outcome to fight.
Ships small, works	good chance	A hundred businesses, \$600K–1M a year, a real company that pays you well and that you own outright. Most founders would take this and most don't get it.
Ships and compounds	real but narrow	The flywheel catches — agencies, chambers, payment volume, the directory. \$5–20M. Requires shipping in the next six months and hiring well.
The big version	unlikely, not absurd	Infrastructure for local business truth, the thing AI assistants query. Needs everything to go right plus timing plus capital. Worth building toward; wrong to plan around.

The distance between row one and row two is not talent, capital, architecture or insight. You have all four. It's whether a specific afternoon happens this month.

07

The one thing

You are not short an idea, a plan, a stack, or a market read.

You are short one running thing.

Everything I'd want to know next about this business — is the remote control viable, will an owner pay \$499, does the email parse survive real mail, is the availability number correct enough to publish, will a bookkeeper refer you — none of it can be answered by thinking harder. All of it is answered by one node, one business, one week.

The documents from today are good and I'd use them. But they're a map, and you've been drawing it for fourteen years. The territory is a phone, a laptop, and a restaurant on 30A that will let you plug something in.

60 Pick the business. Run the test. Film it.

Written as an assessment, not a recommendation to follow blindly — you know this market and this buyer better than I do, and where we disagree the burden is on me, not you. Companions: the Playbook, the Build Spec, the Build Plan, the Parts Catalog.

COMPLETE WORKING RECORD

A NEXT GENT Playbook

Everything from one long working session: the thesis, the machine, the architecture and why, the products, the economics, the go-to-market, the brand, the open decisions, and what to do first.

01

The thesis

You are selling churn absorption. Red Hat's subscription is not access to Linux — it's a promise to absorb upstream churn so an enterprise never has to. Customers renew because upstream never stops moving.

Your upstream is the user interfaces of the apps a small business runs on. Toast redesigns the hours screen. Square moves a button. Google Business ships a new layout. Every DIY automation in America breaks silently and the owner finds out from an angry customer at a locked door.

Yours breaks too — for about six hours.

```
1  a vendor ships a UI change
2  post-action verification fails across the fleet — you know first
3  you re-map that one flow, once
4  signed, pushed to every customer overnight
5  nobody noticed anything happened
```

The flow library is **shared**. Mapping Toast's hours screen costs the same whether you have four customers or four hundred. Marginal cost of the 401st Toast customer is near zero; value to them is identical.

App UIs change far more often than kernels do. That makes this a stronger recurring business than the one it's modeled on.

The fourteen-year insight

The hard part of a directory or a search engine was never the index. It was the data. What changed isn't storage — it's that every business now *receives* its own structured operational data by email, and runs apps that can be operated. **The acquisition mechanism is what was missing.**

The 2013 framing still holds, unchanged, and it's the pitch:

"Nobody cares about 1.8 trillion search results. They have a faucet leak. They need a plumber now."

02

What you're selling

One foundation, many sellable surfaces. A customer can start with a QR menu and end up on the full operating environment without you rebuilding anything.

PLATFORM

A NEXT GENT

The operating environment: identity, workspaces, Constitution, permissions, agents, capabilities, apps, data, automations, devices, proof history. The model underneath is replaceable.

HARDWARE

Ghost

The physical node. Optional — a deployment of A NEXT GENT, not the platform itself. Version one: *the Ghost of Lisa*.

DATA

CyberCheck

Canonical structured business truth with provenance and controlled public/private exposure.

DATA

Quantum Warehouse

Source crosswalks, normalization, history, reconciliation. How you know, where it came from, which source is authoritative.

ECOSYSTEM

VAPPS + App Store

Capability packages. Developers build once against stable capabilities; any business grants access in one tap. 0% to developers.

DIRECTORY

GCR

Local discovery powered by verified, consented, item-level business truth. The consumer side of the same data plane.

Four businesses on one foundation

- **Software** — apps, assistants, websites, booking, menus, reviews, loyalty
- **Hardware / managed computing** — Ghost, Android execution, voice stations, displays
- **Platform / developer** — VAPPS, App Store, SDK, marketplace, APIs, certification
- **Data / network** — CyberCheck, Quantum Warehouse, GCR, partner and chamber networks, agent-accessible business truth

Same identity, same workspace, same Constitution, same capability registry, same execution layer, same verification, same ledger. Four revenue lines, one backend.

03

The machine, end to end

Four machines

WHERE	WHAT LIVES THERE
Workstation	Flow mapping and factory. Maestro Studio, appium-inspector, trailblaze, browser-use, syft, grype, Tekton. Never ships.
Cloud	Quay, Pulp, Katello, Foreman, flightctl, Candlepin, rekor, Zitadel, Postgres, Lago.
Box	bootc OS, Podman, zot mirror, llama.cpp, litellm, ContextForge, uiautomator2, Playwright, Wyoming, SpiceDB, node agent.
Phone	\$40 Android, their SIM, their logins, TextBee, accessibility service. You don't ship it.

One request, all the way through

```
owner texts "post tonight's special"
  ↓ TextBee – phone, real SIM
  ↓ litellm → local model → capability + arguments
  ↓ Zitadel (who) → SpiceDB (may they)
  ↓ risk tier → medium → run and notify
  ↓ execution-router: Facebook has no usable API here → Android
  ↓ uiautomator2 replays the AppMap – fingerprint checked first
  ↓ Playwright reads the public page back – different path than wrote
  ↓ expected == observed → screenshot + post ID → Proof Ledger
  ↓ SMS back: "Posted ✓ verified – ACT-20260913-00814"
```

Every layer is a separate container. Swap llama.cpp for vLLM, uiautomator2 for Artemis, SpiceDB for OpenFGA – nothing above or below changes.

The four ingestion surfaces

- **Email** — every vendor already sends the business daily summaries, receipts and review notifications. Universal, free, and they can't turn it off without breaking their own product.
- **QR menu** — your own sensor. Scan time, table, dwell, what they browsed. No POS generates this.

- **Remote control** — reading state off apps that have no feed.
- **Owner text-back** — the daily toggle. "Catch of the day?" "Still have grouper? Y/N."

NOTE That text thread is the heartbeat. Freshness is what every downstream surface is worth. If they stop replying, the menu, the website, the directory and the phone agent all decay — and stale data poisons faster than no data.

Session reconstruction

```
19:04  scan      table 12
19:04  browse     apps → entrées, 90s on seafood
19:11  order        calamari, shrimp scampi, 2 bushwhackers
20:02  checkout     $94.20
20:15  text         "how was the scampi?"
20:19  review       4.9 — verified, item-level
```

Four sources joined into one **visit**. Toast doesn't know about the scan. Google doesn't know what they ate. You know all of it and the business owns it.

04

Architecture decisions and why

Image-based OS, not package-based

Where the system's state lives — and who assembles it — decides your support cost and what you can promise a buyer.

	REGULAR LINUX	CURATED MUTABLE	IMAGE-BASED
State assembled	on the device	on the device	in your CI
Machines identical	no	roughly	byte-identical
Update unit	~2,000 packages	packages + migrations	1 signed image
Can fail halfway	yes	yes	no
Rollback	none	manual, partial	automatic, invisible
Migration debt	n/a	grows forever	zero
Scales with	machines × configs × versions × hardware		number of SKUs
Subscription gating	hard — whole repo	hard — whole repo	trivial — one pull
10-year promise	sort of	no — rolling release	yes

The row that matters most is the second-to-last. Support cost in the first two columns grows combinatorially; in the third it grows linearly with SKU count. That's the difference between hiring your way to 100,000 customers and drowning at 5,000.

The two rules the runner never bends

- RULE 1

Fingerprint before acting. One or two elements must be on screen before the first tap. If they're not, stop and report DRIFTED. An app redesign fails loudly on the first machine instead of quietly changing the wrong field on four hundred.
- RULE 2

Read the result back, on a different path than you wrote it. Wrote via Android, verify via browser. If the verifier is the executor, you've built self-certification with extra steps. No verify block, no OK.

Publish, don't push

Never build a pipe into the customer's machine. You publish a signed image to a registry; the device polls, checks the signature, and decides. **You cannot be compelled to do something you have no**

mechanism to do. No government, plaintiff, or attacker with your credentials can target one customer, because targeting doesn't exist in the architecture.

Prove it with a **public transparency log** (rekor). Devices refuse anything not publicly logged, so a targeted build would have to be published to the world first. That turns "trust us" into "you don't have to."

BETTER USE Put the transparency log on **constitution versions and AppMap promotions**, not just OS images. Those are the claims that actually need third-party verification.

Take money without taking identity

Billing knows who paid. The update service knows only that *a* valid subscriber asked. Blind-signed tokens (Privacy Pass — deployed in production by Cloudflare and Apple) make the link mathematically impossible, even for you.

"I don't hold your data" stops being a policy that changes when a board changes, and becomes a property of the system. You couldn't produce a user list under subpoena because you don't have one.

The agent is deliberately dumb

Intent → column → answer. No generative step means no hallucinated price. Same strict schema for every business in a vertical means no per-customer prompts, no per-customer tuning, no per-customer regressions — and no training.

The schema is the product. The model is a lookup. Anyone can buy the voice layer for five cents a minute. Nobody can buy columns that are correct right now.

- **Never guess.** Off-schema means a clean handoff plus a text to the owner. Never a plausible-sounding answer.
- **Log every unknown.** After 500 calls across 20 businesses that log *is* the complete schema for that vertical, ranked by real demand. Free roadmap.
- **Show freshness.** Every fact carries an observation timestamp. Stale beyond threshold, it degrades to "let me check."
- **Answer fast, execute slow.** Voice needs sub-800ms. If booking needs remote control (10–20s), confirm immediately, hold the slot against your own inventory, complete in the background.

Local models: the right shape

Disk isn't the constraint — RAM and load time are. An 8B model at 4-bit is ~5GB and 2–8 seconds to load. Eight seconds of "loading the cooking model" is a dead product.

TIER	WHAT'S THERE	SWAP COST
Always resident	1–3B router + embedding model	never unloads
Hot	one strong 8–30B base, permanently in RAM	never unloads
Instant	LoRA adapters — legal, medical, code, cooking, finance	~200ms
On demand	genuinely distinct models — vision, speech, big coding	5–20s, honest UI

50 specializations as adapters cost ~5GB instead of ~400GB, and swap in 200ms instead of 8 seconds. Ship 150–250GB of curated models and adapters, not 1TB.

Hardware implication: unified memory is the only affordable way to give a GPU 64–128GB. AMD Strix Halo (Ryzen AI Max+) with in-tree amdgpu keeps the single-image OS clean — that's an architecture decision, not a benchmark one. Ignore NPU TOPS claims; NPUs are near-useless for LLM inference today.

05

The six unsolved problems

Specified across the documents but not solved, and no repo in the 643-item census solves them for you. These are the ones that will break the build.

1. Capabilities defined by postcondition, not action

`menu.update_price` runs via API on the website, a browser map on Google, an Android route on Toast. If the capability is a *sequence of steps*, you get three unrelated implementations and no way to say whether they did the same thing.

A capability is a state assertion plus a set of executors that attempt to bring it about.

```

capability: menu.update_price
postcondition:
  subject:      menu_item{id}
  predicate:    price == $18.99
  surfaces:     [website, google, facebook, qr_menu]
  observable_by: [api.read, browser.read, android.read]

```

Executors become interchangeable by construction and verification is derived from the contract instead of written per-executor. This is the change that makes the execution-router coherent rather than a switch statement.

2. Verification has three states, and must run on a different executor

- **Propagation delay.** Google Business doesn't reflect a change instantly. Read back immediately and you record a false FAILED. Every capability needs a **settle window** and retry-until-stable before it asserts anything.
- **Partial application.** Price landed on 2 of 4 surfaces. Not VERIFIED, not FAILED.
- **Unsafe re-read.** Some actions can't be probed without cost or side effect.

Result type is VERIFIED | UNVERIFIABLE_YET | CONTRADICTED | PARTIAL, and UNVERIFIABLE_YET is a schedulable state, not an error.

STRUCTURAL The verifier must not be the executor. If the browser map both writes and reads the Facebook page, that's self-certification with extra steps. Write via Android, verify via browser. The independence is the proof.

3. Failure is the unbuilt half

Every document is about proving success. Operationally the money is in what happens on CONTRADICTED. Three things missing from the capability spec:

- **Idempotency keys.** A retry must not double-post, double-charge or double-book. Every ActionSpec carries one; every executor honors it. Without this, retry is a liability.
- **Compensating actions.** A price change applied to 2 of 4 destinations is *worse* than one that failed cleanly — the business is inconsistent and nobody knows. Each capability declares its compensation: roll the two back, or drive the other two forward. Pick one per capability, in the manifest.

- **Escalation thresholds.** Which failure, after how many executor attempts, becomes an SMS rather than another retry. A policy value, not a constant.

4. AppMap versioning — the moat's hard problem

scrcpy and Maestro are commodity. The accumulated map — routes, fingerprints, recovery states, reliability stats per app per capability — can't be bought, and improves for every customer when one breaks. The unsolved part is binding.

- **Pre-flight fingerprint.** Before acting, hash the screen's structural signature and compare to what the map expects. Mismatch means stop and escalate to the AI executor — *not* proceed and hope. Acting on an unrecognized screen is how you change the wrong field.
- **app_version × map_version as the registry key**, not app → map.
- **A promotion pipeline with a gate.** AI discovers the new route → replays in sandbox against a known state → canary on a small slice → promoted to deterministic. The gate criteria decide whether a bad map reaches 400 customers.

Facebook / publish_post / browser task pattern					
success	9,821 / 9,847	verified	99.72%	4.1s	\$0.002
Facebook / publish_post / AI GUI agent					
success	903 / 1,000	verified	90.3%	17.0s	\$0.080

When deterministic reliability falls, route to the AI executor, learn the changed workflow, test it, promote a repaired deterministic pattern. Those numbers seed the execution-router's scoring — use android_world to generate them rather than guessing.

5. CyberCheck needs a conflict resolution policy

Keeping conflicting observations instead of overwriting is the best idea in the documents. But confidence = 0.95 has to come from somewhere.

Observed		
Google	Friday close = 9 PM	observed 5:34 PM
Facebook	Friday close = 10 PM	observed 5:36 PM
Website	Friday close = 10 PM	observed 5:37 PM
Canonical		
Friday close = 10 PM confidence 0.95 last verified 5:40 PM		

- **Per-field source ordering.** Menu prices: POS > website > Google > Facebook. Hours: owner declaration beats everything. A table you write once per field class.
- **Staleness decay.** Forty minutes old outranks six days old; both decay toward unknown.
- **Escalation rule.** Two high-trust sources disagreeing past a threshold is an owner question, not an automatic resolution. Silently picking a winner is how you confidently publish the wrong price.

6. Put the transparency log where it matters

Signed OS images are table stakes. The externally-verifiable claims that actually matter are **which constitution version authorized an action** — with proof that version existed then and wasn't written retroactively — and **which AppMap version was promoted, when, on what evidence.**

That turns the Proof Ledger from something you assert into something a third party can check. It also answers the constitution service's in-flight problem: an action binds to a constitution version at ActionSpec creation and carries it to completion even if the owner amends mid-task, and the amendment is logged with its own timestamp so the sequence is provable.

The policy decision ladder

ALLOW	standing rules clearly permit – continue
ALLOW_WITH_CONSTRAINTS	only this resource, amount, recipient, window
REQUIRE_APPROVAL	human, task-bound, expiring, single-use
DENY	never, regardless of who asks

- No agent may expand its own authority. Possession of a tool, model, credential, device or network connection does not imply permission to use it.
- A model recommendation is not an authorization decision.
- Raw credentials are never implied by permission to perform a capability.
- Agents request by scope and purpose, never reaching directly into stores.
- Execution success is not proof of outcome — postcondition verification stays separate.
- Human approval is narrow, task-bound, expiring and single-use unless the owner explicitly changes the standing rule.

- Even a full-authority agent routes privileged requests through the gateway, so access, actions, data released and resulting proof are documented. Full authority means policy auto-allows within scope — not bypassing the audit path.

06

The data plane

The gate is closed at the vendor. It's wide open at the business.

Toast won't give you an API. But Toast already sends that restaurant a daily summary, a receipt for every order, and a notification for every review — into an inbox the business owns. Nobody can block that without breaking their own product.

Collect at the business, not at the vendor. It works for every vendor at once, including the ones that don't exist yet.

A consented data plane, federated

Not a data aggregator. The business is the holder; it grants scoped access and revokes it. You never own a central pile — which is why businesses say yes, and why "decentralized" is accurate rather than marketing.

ingestion	Tika / Docling parse, langextract structures
mapping	Quantum Warehouse — which field means what, where from
truth	CyberCheck — canonical, with provenance
grant	capability-registry + policy-engine — one-tap approve
wire	MCP and A2A

Developers get answers, never rows

```

developer app → "offer 15% to anyone who ordered scampi in 90 days"
               ↓ A2A
business agent → runs locally against local data
               ↓
returns        → "sent to 312"

```

They never learn who, never learn numbers, never learn what else was ordered. Five things enforce it:

1. **Capabilities defined without data in the signature.** `offer.send_to_cohort` returns a count. `customers.list` doesn't exist.
2. **Developer logic runs in WASM with no network egress** — Extism + wasmtime. Only approved host functions. The code physically cannot phone home.
3. **Scoped, revocable, time-bounded grants** — per purpose, visible in the owner UI, one tap to kill.
4. **A2A as the hop** — capability invocation, not data transfer.
5. **Every access in the Proof Ledger** — which developer, which capability, what came back.

THE LEAK The return value is the attack surface. Enough narrow questions reconstructs the dataset — "how many ordered scampi," filtered a thousand ways, rebuilds the customer list one count at a time, and every query looks harmless. Fix by shaping returns: minimum cohort size (10–25), rate and query budget per grant, aggregates-only for analytics scopes, and enumeration-pattern detection in the ledger. Solved in principle, almost never implemented — doing it properly is a real differentiator.

Machine-readable or invisible

A restaurant's menu as a PDF is invisible to AI. Emit four surfaces on every business page:

- **Schema.org JSON-LD** — Restaurant, Menu, MenuItem, Offer, AggregateRating, OpeningHoursSpecification, Reservation
- **/well-known/agent-card.json** — A2A discovery
- **An MCP endpoint** — for agents that ask rather than scrape
- **llms.txt** — costs nothing

Per-dish AggregateRating backed by verified reviews is the part nobody can fake.

Agent analytics — a product nobody sells

This month, AI agents asked about your business 1,247 times

hours / open now	412	
menu items	338	
availability tonight	201	
gluten-free options	94	← no data here
parking	67	← no data here
dog friendly	41	← no data here

Google Analytics says someone visited. This says what they wanted to know. For the owner that's the renewal. For you, aggregated across 200 businesses, it's a demand-ranked schema roadmap.

Public JSON-LD is free and anonymous. The richer MCP/A2A surface requires identification — live availability, item ratings, the ability to book. Good trade for the agent, and the agents that matter end up identified.

07

Products and verticals

Real-time availability — the biggest one

A charter doing \$400K/yr through FareHarbor at 20% pays **\$80,000 a year in commission**. They pay it because their own website can't show real availability. Shift 30% to direct and they save \$24,000. Your \$499/mo is \$6,000.

SALES CALL 4:1 return on one line item, provable from their own statements. Not "saves you time" — "here's twenty-four thousand dollars."

declared capacity	6 jet skis	
FareHarbor email	2 booked 10:00-12:00	→ 4 left
Booking.com email	1 booked 10:30-12:30	→ 3 left
cancellation email	1 released	→ 4 left

FareHarbor doesn't know about the Booking.com reservation. You're the only thing that sees all channels at once.

- **Email lag** — confirmations arrive 30s–2min late. Hold a safety slot; show 4 when you have 5.
- **Walk-ins generate no email** — one-tap decrement on the owner's phone or the wall screen. Design it before launch.
- **Template churn** — fingerprint the format, fail loudly, re-map once, push to everyone.
- **Drift** — reconcile by reading real platform state through the remote control on a schedule. Your verification layer applied to inventory.

Verified item-level reviews

The box reads the POS. It knows who, when, and exactly what. The review is tied to a real receipt with real line items — not "4 stars, good food."

Chicken Alfredo	4.7	• 312 verified orders
Shrimp Scampi	4.9	• 188 verified orders
Calamari	4.2	• 241 verified orders
Bushwhacker	4.8	• 906 verified orders

That's a dataset, not a feature. It feeds the QR menu, the public page, the owner's margin decisions, the directory, and recommendations. Accumulated across a region it's unrebuildable.

The ask writes itself: *"Hey Bob — how was the chicken alfredo?"* gets answered. *"How was your visit?"* doesn't.

- **Host it first.** Reviews live in CyberCheck and on your surfaces. Syndication out is downstream and optional. Build it the other way and you're a review-request tool that owns nothing.
- **Never comp for a review.** The instant there's an incentive the ratings are worth less, and being trustworthy is the entire value. A real 4.2 beats a fake 4.8.
- **Publish only on completed visits** — verified by POS ticket or QR scan.

The phone assistant

Every AI phone agent on the market fails on **data, not voice**. They're grounded on a static menu PDF and a hardcoded hours string from setup day, so they quote March's price and offer a booked slot.

The business turns it off in three weeks.

Voice is \$0.05/minute and commodity. You own the half nobody has: live availability, current prices from the POS, today's hours, what's 86'd.

A restaurant misses 20–40% of calls during rush. An answering service is \$200–500/mo and can only take a message. Yours books revenue.

Remote phone access

Most small business owners carry an iPhone and physically cannot run Android-only business apps. The handset lives in the cradle at the shop; they see and touch it from their iPhone anywhere. **That's a reason to buy the box before any AI does anything.**

ws-scrpcy	browser front-end, screen + touch, works in iOS Safari
scrcpy + adb	over the network instead of USB
headscale	phone app → box → handset, point-to-point encrypted

Same connection the agent uses, so human takeover is free — and what they do by hand becomes a new AppMap.

Multi-account ceiling: Android work profile or OEM app cloning handles two instances comfortably. Five separate Airbnb logins needs more handsets — design the cradle for two or three from the start.

- Device-bound auth, enrolled once, keyed to the box
- Nothing on your servers — the relay is the mesh
- Every session in the ledger, visible to the owner
- A physical kill switch on the box

SMS from their own SIM

Every SMS marketing platform's margin is the per-message markup. Yours is zero — it's the customer's SIM on their plan. And the box already has the list, because it's reading bookings, POS, reviews and calls.

"No per-message fees, ever" is a sentence no competitor can say.

WATCH Carriers throttle bulk sending from a person-to-person line — low hundreds per day before deliverability degrades. Split it: own SIM for conversational, 1:1, approvals and small batches; a registered A2P route behind the same interface for real blasts. Test the actual daily ceiling before putting a number in marketing.

Artist song requests

QR on the stage → landing page → request + tip → confirmation email parsed → identity tied to request. Zero new architecture.

Fifty artists playing 4–6 nights a week to 100–300 people is roughly **30,000 impressions a week**, in venues, with people holding their phones in spending mode. Give the artist product away entirely — they're the channel, not the revenue.

FLYWHEEL The QR stands in a bar every Friday. The owner watches it work and asks the musician what it is. **Sign artists to sell bars.**

Don't process the tips. Let them use their own Venmo or Cash App — free between people, "you keep 100% of your tips," and the confirmation email still gives you the tie-together. Same mechanism as everything else.

Three consumer queries nobody can answer

- "Who has crab legs nearby?" — menu data is in PDFs, behind logins, or stale on Google
- "Who's playing live tonight?" — Bandsintown and Facebook events are stale and incomplete
- "Two bed two bath at Silver Shells, these dates" — nobody lets you search by *complex*, which is how beach markets actually search

Same machine answers all three. The third is the highest value — the transaction behind it is \$2,500, not \$90.

Cross-business recommendations

This person had the grouper at A, the scampi at B, rated both — recommend the snapper at C. Yelp has reviews but not verified purchases. Google knows you visited but not what you ordered.

Useless until a market is dense. Twenty restaurants in one town works; two hundred across three states is noise.

08

Economics and pricing

Price against what you replace, not against a laptop. Your comps are Yext (\$199–999/yr), Birdeye / Podium / SOCi (\$250–500/mo), a part-time VA (\$1,500–3,000/mo), an answering service (\$200–500/mo). Those manage listings. You execute.

LINE	PRICE	NOTE
Software	\$249 / mo	Their hardware, their phone. Core flows, approval queue, SMS interface.
Ghost	\$499 / mo	Box + handset, \$0 down, 24-month term. Unlimited flows, voice, phone line, priority support.
Multi-location	+\$299 / mo	Each additional location.
Regulated	\$799 / mo	Clinics, firms. Retention controls, compliance logging, SLA.
Onboarding	\$1,500–3,000	One-time. Connect accounts, map flows, configure, public pages. Filters tire-kickers.
Custom AppMap	\$500–2,000	Weird vertical app. You map it and maintain it.
Developers	0%	Say it loudly. Apple takes 30%. It's marketing.

Bundle the hardware. \$2,495 up front gets "let me think about it." \$499/mo gets a yes. Toast, ADT and every successful small-business hardware play does it this way.

PRICING Start at \$499, not \$249. Same sales call, same support load. If nobody pushes back you priced too low, and you can always discount the first ten for proof. You can't raise on people you signed cheap.

Transactions — 0% direct, a cut on what you bring

BOOKING CAME FROM	YOU TAKE
Their own website	0% — they just pay Stripe 2.9% + \$0.30
Your directory	X% — you keep it
Partner directory	X% — split with them
Revenue you created	10% — refills, last-minute, cancellations filled
FareHarbor / Booking.com	nothing. Don't chase it, don't invoice.

"I don't take a cut of your existing business. Only what I bring you." Structurally aligned instead of taxing them — and the partner split makes other directories *want* to carry your data, which funds distribution.

NEVER A cut you have to invoice for is the worst of both worlds. If the booking happened on FareHarbor, the money lands in the business's account and you're sending invoices to a charter captain in February. Own the rail for bookings you create; take nothing on the rest.

Payment facilitation — the real long-term line

Standard model. Toast, Square, Shopify Payments all negotiate wholesale and resell blended. Stripe Connect supports it natively — `application_fee_amount` is your spread.

	YOUR COST	YOU CHARGE	SPREAD
Card present	~1.8–2.1%	2.9%	0.8–1.1%
Online booking	~2.4–2.7%	2.9% + \$0.30	0.2–0.5%

1.9% all-in isn't reachable for card-not-present. Interchange alone is 1.8–2.3% + \$0.10 and nobody negotiates below it. What you get at volume is interchange-plus, roughly IC + 0.4–0.6% + \$0.10–0.20.

- **~\$1M/yr** — a conversation, not a rate
- **~\$12M/yr** (\$1M/mo) — real interchange-plus on the table
- **\$50M+/yr** — genuine leverage, custom terms
- **\$200M+/yr** — direct processor or becoming a payfac is worth the compliance burden

HIDDEN COST Dispute liability. With destination charges the **platform** is liable by default, plus negative balances when a business closes holding deposits. Budget 0.1–0.3% of volume for disputes and reserves, and set the liability structure deliberately. That's what turns a 1% spread into 0.7%.

Why 0% is the highest-leverage price in the model

Shopify charges 0% transaction fee *if* you use Shopify Payments. The free tier isn't generosity — it's volume acquisition, and the processing spread is the business.

Free on direct → they consolidate onto your rail → you get 100% of their volume → volume buys the rate → the rate is the revenue. Every competitor charging a platform fee on direct bookings leaves that consolidation on the table.

Target high-ticket for volume

CUSTOMER TYPE	AVG TICKET	ANNUAL VOLUME EACH
Vacation rental manager (40 units)	\$2,500/wk	\$2–4M
Hotel / condo property	\$200–400/night	\$1–5M
Charter fleet (6 boats)	\$800/trip	\$800K–1.5M
Marina	varies	\$1–3M
Single charter	\$800	\$200–300K
Restaurant	\$60–90	\$1–3M, tiny tickets

Twenty vacation rental managers is \$60M/year in volume. Twenty sales calls to serious negotiating leverage, versus 500 restaurants.

Split the targeting: property managers and charter fleets for payment volume, restaurants for data density and the directory. Different sales motions — don't run them as one. Watch seasonality (Gulf Coast can be 5:1 summer to January — negotiate on trailing-twelve after peak) and concentration (no vertical past ~35% of volume).

Loyalty points

A single-business loyalty card is worthless to someone who visits once. **Regional points** — earn on a charter, spend at a restaurant — is something no individual business and no OTA can offer, and it's the first thing that gives a tourist a reason to use your directory instead of Booking.com.

- **Clearing** — the business giving the discount isn't the one that earned it. You're already in the middle of the transactions. Decide up front: pool buy-in, or earning business funds the liability at issue.
- **Terms and expiry from day one.** Unredeemed points are a liability on paper and revenue in practice. Retrofitting terms is a fight you don't want.
- Keep v1 a points ledger and a redemption code. Don't build a currency.

Referrals — the version nobody can fake

Affiliate content is drowning in AI slop. A creator video tied to a verified booking *and a verified completed visit* can't be manufactured. They can fake the face; they can't fake the receipt.

creator link → booking → visit completed → verified → payout

KEY Pay on verified completion, not on booking. Every affiliate network pays on a conversion they can't see. You're standing at the end of the transaction. That kills fake bookings, cancel-after-payout, and owner self-dealing in one move — and no competitor can claim it.

Decide attribution before launch. Ambiguous case — found on your directory, booked direct two days later — **default to 0%**. Costs a little revenue, buys the thing the model runs on. Write the policy down and show it to them.

White label — the GoHighLevel model

Don't sell 100,000 businesses. Sell agencies with 20 clients each. They do sales, onboarding, support and customization; you run the platform. **And it closes the biggest hole in the direct model — support cost at scale.**

Agency pays you	\$199-249 / box / mo wholesale
Agency sells at	\$499-699
Agency keeps	\$250-450 / mo per client, plus install and
You get	predictable revenue, zero support load, zero CAC

An agency with 40 clients makes \$12-18K/month off your platform. Your trained rental-ops reps are exactly the people who become those agencies. And unlike a GHL agency they get a service business too — installs, phone setup, flow mapping — which makes them stickier.

Revenue math

- 100 customers × \$499 = **\$599K ARR** plus ~\$200K onboarding in year one
- 250 = **\$1.5M**
- 1,000 = **\$6M**
- 200 businesses × \$16M/mo processed at 0.7% net = **~\$1.3M/yr**, growing with their revenue rather than your headcount

METRIC

Flows per customer is your only real retention number. Two flows and they churn. Fifteen and they've moved their operations onto your box and they're not leaving. Instrument it from customer one.

Go to market

The email forward is the entry point, not the box

"Here's an email address. Forward your booking confirmations to it. That's it."

Five minutes, no hardware, no install, no phone, no card. With just that you deliver real-time availability, verified bookings, a public page and widget that work, and the beginning of their structured data.

That's the free tier. The box is the upsell for *execution*. It collapses CAC to nearly nothing, kills the developer chicken-and-egg, and means your installed base can be ten times your paying base — every forwarding business in the directory, feeding the data plane, already half sold.

Scale by vertical, not by geography

The schema is the unit of work, not the map. A rental manager in Destin, Gulf Shores and Panama City are the same schema, same flows, same emails. Once the vertical is built, customer 2 through 200 cost nothing new regardless of where they are.

Density still matters for the consumer directory and the developer market — a developer needs to see an addressable market — but it arrives as a side effect of going deep in one vertical first.

Channels, ranked

1. **Multi-unit operators.** A franchisor with 70 locations isn't a customer, it's a distribution event. One decision-maker, 70 boxes. Restaurant groups, rental management companies, marina operators. Highest leverage by far.
2. **Chambers of Commerce.** 300–800 members, wants to offer value, has no technology. Give them a live directory powered by member data; every member becomes a warm introduction. One deal is a city.
3. **Artists.** Free product, 30,000 QR impressions a week, and every code sits in a venue you want to sell.
4. **Agencies.** White label. Your trained reps become the agencies.
5. **Direct.** The first few hundred, by you, because platform flywheels never bootstrap.

Sales comp — where these programs die

- **Split commission on retention.** Half at install, half at 90 days retained. Pay full on signing and reps sell to anyone with a pulse.
- **Residual while the account lives.** Door-to-door people understand residuals. A rep with 60 accounts defends them and never leaves.
- **Let them earn on payment volume.** A rep who signed a rental manager in year one is still earning on \$3M of processing in year four. No other sales job can offer that.

The developer pitch

"You can vibe code this. You just need access."

LLMs made code free. Nobody can generate a Toast confirmation email, a phone with the owner's logins, real availability across four channels, or a business that said yes. Code stopped being scarce. Access didn't.

Keep the primitive set small. Five calls build the song request app:

<code>page.publish</code>	a public page with a QR
<code>payment.observe</code>	watch for a confirmation
<code>identity.match</code>	tie the payment to the request
<code>notify.send</code>	tell the artist
<code>data.write</code>	store it in their workspace

Same five build a tip jar, feedback form, waitlist, deposit collector, raffle, review request, table-side service call, lost-and-found, event RSVP. **That's the recruiting demo** — one page, five calls, nine apps.

ALTITUDE Verbs, not rows. Too high-level (`song.request`) and you're building apps, not a platform. Too low-level (raw email) and you've handed over the data. `payment.observe` returns "a payment arrived matching these criteria," never the inbox.

Build the app that can't exist anywhere else

"Plan my Saturday" → checks a charter's real availability, checks a restaurant's tables, books both, one confirmation. Two businesses, both consenting, neither sharing data with the other. No POS, OTA or directory can do it — they'd each need a partnership. You need two businesses that said yes.

Compounding, not accumulating

An app that only consumes capabilities adds one app. An app that *publishes* one makes every future app easier. Pay component authors from the apps built on them — your capability registry knows exactly which VAPP called which, which nobody else can measure. Publish the formula and the payouts; a verifiable split is a differentiator, a vague promise is a blog post.

10

Brand and narrative

The story

"Jobs took Unix, hid where it came from, and closed the ecosystem. I'm taking Linux, showing where it came from, and opening it."

NeXTSTEP was Mach plus BSD. Apple bought NeXT in 1997, it became Rhapsody, then Mac OS X. Every Mac and iPhone today runs on a foundation taken from Unix and covered up.

Most companies invent a narrative after the fact. You have a real one with forty years of lineage, and the logo carries it without explanation to anyone who knows.

The Ghost of Lisa

Lisa shipped January 1983 at \$9,995 — about \$32,000 today — with protected memory, cooperative multitasking, a document-centric interface, drag and drop, system-wide undo, the trash can, the menu bar, an integrated suite with a shared clipboard, and Gantt charts on a personal computer.

FOR THE AD **The Macintosh didn't get protected memory or real multitasking until Mac OS X in 2001.** Apple shipped it in 1983 then went backwards for eighteen years to hit a price point. The industry chose cheap-and-worse and took two decades to climb back.

~2,700 unsold Lisas were crushed and buried in a Logan, Utah landfill in 1989. In January 2023, on the fortieth anniversary, Apple released the Lisa source code through the Computer History Museum. **The machine they buried is now open source.**

Lisa	right, unaffordable, buried
Macintosh	affordable, closed
Ghost of Lisa	right, affordable, open

Lisa failed on access, not technology. You're not early — every component you need is mature and nearly free: a \$40 Android, a \$500 box with 128GB unified memory, models that run locally, forty years of Linux, Stripe. None of that existed five years ago. And naming v1 after the famous failure means nobody can accuse you of overpromising.

"It's not AI"

Everyone is slapping AI on everything at the exact moment small business owners stopped trusting it. "AI-powered" reads as *risk* to a 55-year-old charter captain.

"It's not AI. It does exactly what you'd do, and shows you proof it did it."

And it's literally true — the agent is deliberately dumb, reads a column, replays a recorded path, screenshots the result. Keeping AI out of the patent claims is right for a second reason: claims about a concrete remote-control method sit outside the hurricane of AI prior art.

The 1984 remake

Make the antagonist the **category**, not Apple. Big Brother is the gatekeeper — every AI company racing to own your computer and rent you access to your own business. Bigger target, more sympathetic, less exposure.

The line that does the work: **they want your business to live on their servers. Yours lives on your counter.**

Ending on the screen shutting off rather than the hammer is the better gesture — power through withdrawal rather than violence, and it maps to the product: the box is on your counter and the switch is yours.

Logo notes

- **Keep the cube.** Paul Rand's NeXT mark, and the cube is literally the product you ship. Most logos never get one idea that good.
- **Keep Tux** — it's the thesis, Jobs hid the Unix and you're showing the Linux — but **redraw it flat**. Glossy 3D cartoon against Rand-era geometry looks like two logos stapled together.
- **Build a standalone mark.** The cube alone, working at 32px, for favicon, app icon and the badge on the box lid. Rand's original reduces; yours needs the same discipline.
- **Replace the typeface.** The thin geometric sans is generic next to Rand's lettering. The box is industrial; the type should agree.
- **Buy every spelling** — anexgent, a-next-gent, anextagent, nextgent. Spoken it sounds exactly like "an agent," which is the joke and the discoverability problem. \$200 now, expensive later.

Stealth, resolved

Quiet about the mechanism, loud about the outcome. Sell "your hours update everywhere, your website shows real availability, your phone gets answered." No blog post about accessibility-service-driven remote control, no architecture diagram in the deck.

That resolves cleanly against open-sourcing, which otherwise contradicts stealth:

- **Open** — the platform, contracts, OS, SDK, capability spec. This recruits developers.
- **Not open** — the AppMap library, the fingerprint and repair engine, the per-vertical schemas. The accumulated operational knowledge the subscription buys.

Secrecy buys 6–18 months and costs every customer, developer and chamber deal in that window. Your real defenses — AppMap library, verified data, installed base, payment volume — all compound with speed, not silence. And your own content strategy guarantees the idea gets out anyway.

11

Competitive position

Why incumbents structurally cannot follow

- **Google** can't tell businesses to stop depending on Google Business Profile
- **Toast / Square** can't say "book direct, skip our fees"
- **FareHarbor / Booking.com** — commission is the company
- **Yext** can't stop charging for distribution; distribution is the product
- **Every seat-based SaaS** can't move to "it runs on your box" without torching ARR per customer

Not slowness — anti-correlation. Following you costs them their P&L.

REAL THREAT Not big tech. A well-funded fast follower with **no legacy revenue** — a team with \$5M and no conflicts. The answer is speed and accumulated data, which argues for 200 customers fast rather than staying quiet.

Omarchy — what it proves and what to avoid

DHH built roughly this product and deliberately declined to build this business. Arch + Hyprland + Quickshell, MIT, ~457 shell scripts, 249 theme files, a 51-chapter manual. Under the Omacom Foundation with ~\$18.5M in pledges. **No subscription, no hardware** — buyers get sent to Dell and Framework.

- **Steal:** the manual quality, theming as a product feature, the single-namespace CLI, and the agent-native positioning.
- **Avoid:** mutable Arch. 116 migration scripts are config-drift management — proof the state still lives on the customer's laptop. And `install/hardware/` is ~30 per-machine fix scripts, the infinite hardware matrix as a directory listing. Your certified SKUs delete that folder; read it as a components-to-avoid list.
- **Fatal for your model:** Arch is a rolling release. You cannot sell "supported until 2036" on top of it.

Home Assistant — the closest living template

Immutable appliance OS with A/B boot slots and RAUC OTA, shipped to millions of non-technical people. And the business is yours: Nabu Casa sells the hardware and a cloud subscription while the Open Home Foundation holds the project, everything Apache-2.0.

Somebody already ran your experiment and it worked. Read their update-system docs before committing to your own OS architecture.

Solid — the one that didn't work, and why

Berners-Lee's decentralized data project is your thesis exactly: personal data pods, apps request scoped permission, data stays with the person. From the guy who invented the web.

- **No killer app** — it asked people to move data for an abstract benefit. Nobody moves for principle.
- **No business model** — a standard looking for a company.
- **Too abstract** — "own your data pod" means nothing to a charter captain.

You have all three. Nobody has to understand decentralization to buy "my website shows what's actually open." That's the gap between a white paper and a company.

Capital — the conflict problem

A fund holding positions in POS, listings, booking, reputation or local search has a structural reason to soften what you do — not by blocking, but by slowing, redirecting, or "introducing" you to a partnership that neuters the direct-booking play.

- **Diligence the fund before the term sheet.** Ask directly what they hold in those categories. A straight answer is diagnostic either way.
- **Local money is strategically better, not just easier.** No portfolio conflict, they know the market, they're customers and references too, and they'll never tell you to go partner with Toast.
- **Don't sell control to solve a cash problem.** Revenue-based financing, customer prepayment, hardware deposits. At \$499/mo with real retention you can fund a lot off the business.
- **Information rights over board seats** for early money.

If the goal is genuinely that it outlives you and can't be captured, the structures exist — Open Home Foundation, Omacom Foundation, Linux Foundation projects, steward-ownership and purpose trusts. A verbal intention doesn't survive a term sheet; a trust deed does. But you can't decentralize something that doesn't exist — build it, get to self-funding, then put the governance around it from strength.

Traction before the call

- 20–30 **paying** businesses, not pilots
- Three months of retention — churn as a number, not a hope
- One ROI case with their statements — the charter that saved \$24K
- One vertical schema done, so customer 200 costs what customer 20 did

Six-month sprint, not a two-year build, and all of it in the first vertical. The thing that makes you fundable is the thing that makes you not need it.

12

The stack

Primary picks by layer. Everything here is open source and most of it was built to compose — Foreman with Katello with Pulp with Candlepin, Backstage with Keycloak, Sigstore with OCI registries. The full component list with alternates lives in the parts catalog.

LAYER	PRIMARY	ALSO ON THE LIST
OS image	bootc + ostree + greenboot + bootc-image-builder	rauc, Mender, Ubuntu Core, Omarchy (donor)
Node runtime	Podman + Quadlet, systemd	MicroShift for multi-service apps
Registry	Quay (cloud), zot (box)	Harbor
Content + channels	Pulp, Katello	Koji
Fleet	Foreman, flightctl (Edge Manager)	FleetDM, Argo CD
Entitlement	Candlepin	IBM License Service model for metering
Signing + trust	cosign, rekor , TUF	Skopeo, go-containerregistry
Supply chain	syft, grype, trivy, scancode, trufflehog, Conforma	CycloneDX / SPDX
Factory	Tekton, Buildah	Konflux
Identity	Zitadel or Keycloak	Backstage ships Keycloak integration
Authorization	SpiceDB or OpenFGA	OPA for contextual policy
Secrets / certs	OpenBao, cert-manager	
Data	Postgres, pgvector, SQLite + sqlite-vec	Meilisearch, OpenSearch, qdrant, Supabase
Events	NATS (box)	Kafka (cloud scale), Flink
Agent runtime	Hermes or BeeAI, behind your contract	OpenJarvis, QwenPaw, OpenClaw, Goose
Protocols	MCP + A2A	ACP is merged into A2A — don't invest
Gateway	IBM ContextForge	MCP Registry

LAYER	PRIMARY	ALSO ON THE LIST
Model routing	litellm, llama-swap	Llama Stack (build to Responses API)
Inference	llama.cpp (box), vLLM (bigger)	Ollama for dev, LocalAI, LLM Compressor
Models	Granite, Granite-Guardian	InstructLab, huggingface_hub, ROCm
Android	ADB, uiautomator2, scrapy, TextBee	Maestro + Studio (factory), trailblaze, appium-inspector, Artemis, mobile-mcp, AgentCPM-GUI, android_world
Browser	Playwright, playwright-mcp	browser-use, browser-harness, Stagehand, Steel, Browserbase
Linux desktop	AT-SPI	computer-use-linux, cua, ydotool
Workflow	Temporal, Event-Driven Ansible	Restate, Ansible, AWX, Node-RED
Integrations	Activepieces, Composio	n8n is not open source — Sustainable Use License
Voice	Wyoming, openWakeWord, silero-vad, faster-whisper, Kokoro	pipecat, LiveKit, Fonoster, hassil, speeches. Piper archived Oct 2025.
Files / data	Tika, Docling, langextract, magika	aifs, mem0 / letta, schema-dts
Sandboxing	Extism + wasmtime, gVisor	bubblewrap, Firecracker, microsandbox, OP-TEE
Verification	Playwright assertions, pixelmatch/odiff, ffprobe, qpdf	JHOVE
Packaging	Flatpak, PackageKit, AppStream	Skilla, xAI plugin-marketplace (donor), Backstage

LAYER	PRIMARY	ALSO ON THE LIST
Interfaces	Next.js, Flutter, cage (kiosk), Norigin (TV)	shadcn, Puck, JSON Forms, distrobox as the escape hatch
Networking	headscale / WireGuard	Tailscale, NetBird, SAM, Sunshine + Moonlight
Commerce	Stripe Connect, Lago	Kill Bill, dub
Backup / firmware	restic, Litestream, fwupd + LVFS	borg, rclone, snapper

What you own — nobody upstream does these

control-plane · identity / organizations / workspaces · capability-registry · ActionSpec · Constitution · policy-engine · credential-broker · agent-gateway · agent-contract and runtime adapter · model-router contract · execution-router · executor contracts (android, browser, linux, api, workflow, human) · AppMap spec and learn-once compiler · appmap-runtime · verification contract · evidence · Proof Ledger · VAPP manifest · marketplace semantics · certification definition · CyberCheck · Quantum Warehouse · exposure rules · device-manager · node-agent · updater and release-channel semantics · trust-service · recovery · sync · migration · operations · event-service · handoff-service · commerce-service · data-governance · edge-service · owner-ui · platform-admin · sdk · cli

RULE One job, one seam, one live implementation. Wrapper depth of exactly one — your contract calls the implementation directly. Anything marked ALTERNATIVE sits behind the same contract as the primary, never alongside it. Factory tools never ship. MCP and A2A are **projections** of your contracts, not layers under them.

13

IBM / Red Hat findings

Take the protocols and the models. Treat the "platforms" as reference designs. Refuse anything that hard-requires Kubernetes. The AI layer is overwhelmingly Apache-2.0 / MIT with real patent grants — much safer ground to build a Red-Hat-style business on than RHEL itself was.

COMPONENT	LICENSE	K8S?	VERDICT
BeeAI framework	Apache-2.0	no	Take — core agent runtime. LF AI & Data governed. RequirementAgent, tools, sandboxed code interpreter, memory, pause/resume, A2A adapters.
MCP + A2A	Apache-2.0 spec	no	Take — MCP is agent → tool (vertical), A2A is agent → agent (horizontal). ACP merged into A2A, Aug 2025 — legacy, don't invest.
ContextForge	Apache-2.0	no	Take — federates MCP + A2A + REST/gRPC. PyPI or plain Docker, air-gap capable, no telemetry.
Granite (all)	Apache-2.0	no	Take — published training-data disclosure and IP indemnification through Red Hat.
Granite-Guardian	Apache-2.0	no	Take — drop-in guardrail model, served on vLLM, called by your policy-engine.
Docling	MIT	no	Take — best-in-class document parsing, has an MCP server.

COMPONENT	LICENSE	K8S?	VERDICT
vLLM	Apache-2.0	no	Take — UC Berkeley origin, community governed, all major hardware backends, standalone container with an OpenAI-compatible API. Red Hat is the leading commercial contributor — this is literally the business you're copying.
LLM Compressor	Apache-2.0	no	Take — quantization for models you ship.
InstructLab	Apache-2.0	no	Take — laptop-capable fine-tuning and synthetic data. The customer-facing "improve it with your data" hook.
Llama Stack	MIT	no	Take as the API-server option. Build to the Responses API — the agents API is being deprecated.
KServe	Apache-2.0	yes	Reject for the box. "No Knative" ≠ "no Kubernetes."
llm-d	Apache-2.0	yes	Reject for the box — explicitly Kubernetes-native.
TrustyAI Guardrails Orchestration	Apache-2.0	yes	Every deployment path needs OpenShift cluster-admin. Run Granite-Guardian on vLLM instead — same detector models, none of the overhead.

COMPONENT	LICENSE	K8S?	VERDICT
OpenShift AI / Open Data Hub	mixed	yes	Reject. Mine the reference architecture (vLLM → Llama Stack → MCP → guardrails → RAG), reimplement with standalone containers.
watsonx Orchestrate	proprietary	—	Reject — the funnel. The ADK is a CLI into a paid platform; you need watsonx.ai or Orchestrate credentials to install Developer Edition. Copy the Agent Catalog concept for your App Store.

Watson is not one thing

Most of what IBM ships under that name is genuinely open and already in your list — Granite, Granite-Guardian, ContextForge, BeeAI, Docling, InstructLab, FMS-Guardrails, the watson-developer-cloud SDKs (Apache-2.0), watsonx-developer-hub, watsonx-data.

Commercial: **watsonx Orchestrate the platform**, watsonx.ai, watsonx Assistant.

The distinction that matters for your model: the watson-developer-cloud SDKs are Apache-2.0 *client libraries for paid cloud APIs* — open code, metered service behind it. Different from Granite or BeeAI, which run on your own box with nothing behind them.

Sandboxed execution — the one place the best tool isn't IBM/Red Hat

BeeAI ships a code interpreter, but the strongest self-hostable options are **microsandbox** (Apache-2.0, libkrun microVMs — libkrun is itself a Red Hat library — fully local, sub-200ms starts) and **E2B self-hosted** (Apache-2.0, Firecracker). MicroVM isolation is stronger than plain containers. Don't be steered to a weaker tool for brand consistency.

DEAD Daytona went closed source June 2026 — the former AGPL repo is archived. Don't build on it. Piper archived October 2025 — still in every tutorial, use Kokoro.

Seams and duplication to know about

- **Two agent runtimes, one orbit.** BeeAI (IBM) and Llama Stack (Meta + Red Hat) overlap. Pick one — framework-in-code or API-server — not both.
- **Two guardrail paths.** Granite-Guardian (model) vs TrustyAI Orchestrator (K8s service wrapping the same models). Same detectors, different packaging.
- **Two protocols that were three.** MCP, A2A, and ACP — now merged in.
- **BeeAI risk.** Pre-1.0 (0.1.x) with an explicit IBM notice that they may not maintain it. "Production-ready" in the README is marketing. Mitigated by Apache-2.0 + LF governance — you can fork and own it — but budget for that.

Licensing and trademark reality

- You can use vLLM, Granite, BeeAI freely. You cannot call your product "Red Hat" anything, and Red Hat's container images and RHEL bits carry subscription and trademark constraints — what CentOS/Rocky/Alma had to strip.
- Use **UBI or CentOS Stream** base images, not entitled RHEL images.
- Post-June-2023, CentOS Stream is the sole public repository for RHEL-related source; RHEL source proper is behind the paid Customer Portal and the subscription agreement discourages redistribution. This affects your OS base decision, not your AI stack.
- **Open in name only:** Red Hat AI Inference Server (support needs a subscription; the vLLM underneath is free), RHEL AI (Granite is Apache-2.0, the platform and indemnification are paid), Ansible Lightspeed (connect-service is self-hostable, the hosted model is commercial).

NOTE Treat the paid tiers as a blueprint for *your own* future product, not something to consume. vLLM → Red Hat AI Inference Server and Granite → RHEL AI are literal templates for "take OSS, harden, certify, indemnify, subscribe."

14

Modularity rules

One job → one seam → one live implementation. Wrapper depth of exactly one. Anything ALTERNATIVE sits behind the same contract as the primary, never alongside it. Factory tools

never ship. MCP and A2A are projections of your contracts, not layers under them.

The nine collisions found in the current stack

1. **Model routing is four shells deep.** `model-router` → `litellm` → `ollama` → `llama.cpp`.
Ollama is a wrapper over `llama.cpp` — two redundant hops. Go `model-router` → `litellm` → `{llama.cpp | vLLM}`; ollama stays on dev machines.
2. **Three ways to tap a button.** `mobile-mcp` wraps `uiautomator2`/`adb`, Maestro wraps `adb`, Artemis is a separate agent. One `AndroidExecutor` contract; **uiautomator2 is the only runtime implementation on the box**. Maestro, `trailblaze`, `appium-inspector` are factory-only. `mobile-mcp` is an MCP projection. Artemis is evaluated as a *replacement*, not an addition.
3. **Two agent runtimes.** Hermes and BeeAI do the same job. The contract makes the other swappable; running both makes neither.
4. **Two gateways.** `ContextForge` is *transport* (MCP/A2A/API federation). Agent Gateway is the *policy envelope*. Say it explicitly or they collide.
5. **Two identity systems.** Zitadel vs Keycloak. Tiebreaker: Backstage ships a Keycloak integration.
6. **Two authorization engines.** SpiceDB vs OpenFGA — both Zanzibar.
7. **Two event buses.** NATS on the box, Kafka only if cloud volume forces it. Never both on a node.
8. **Four search/vector stores.** `pgvector` for embeddings, Meilisearch for public text, `qdrant` stays out until vector load justifies a second store, `OpenSearch` is cloud log scale only.
9. **Three update-trust layers.** `bootc` is *delivery*, `cosign+rekor` is *signature* — different slots. TUF vs `cosign` genuinely overlaps; TUF earns its place only for key rotation and role delegation. `Mender` vs `bootc` is the same slot and has been open across three document generations.

The thirteen wires — everything plugs into one

Nothing gets forked and nothing gets picked. Everything below already speaks one of these, which is why they connect without integration work.

WIRE	WHAT SPEAKS IT
OpenAI-compatible HTTP	llama.cpp server, vLLM, ollama, LocalAI, llama-swap, litellm, Llama Stack, TGI — plus every cloud provider
MCP	ContextForge, playwright-mcp, mobile-mcp, docling-mcp, Composio, Activepieces, MCP Registry, your VAPPs, CyberCheck's public surface
A2A	A2A project, BeeAI (Agent Cards), Hermes, OpenJarvis, external agents, someone else's ChatGPT or Claude
ADB	uiautomator2, Maestro, scrpy, Appium, trailblaze, Artemis, mobile-mcp, DroidRun — all on the same connection to the same handset, simultaneously
CDP	Playwright, browser-use, browser-harness, Stagehand, Browserbase, Steel
AT-SPI	computer-use-linux, cua, ydotool, wtype
Wyoming	openWakeWord, silero-vad, faster-whisper, whisper.cpp, Kokoro, HA Assist, pipecat
OCI registry	Quay, zot, Harbor, skopeo, buildah, oras, cosign, rekor, bootc, Flatpak (OCI mode), your flow packs, your VAPPs
OIDC / OAuth2	Zitadel, Keycloak, Backstage, Cockpit, Grafana — run either or both, federated
Postgres wire	Postgres, pgvector, Supabase, Drizzle, Pulp, Candlepin, Keycloak, ContextForge, Temporal, Backstage
Events / CloudEvents	NATS, Kafka, Event-Driven Ansible, Temporal, Activepieces, Composio
Zanzibar check API	SpiceDB, OpenFGA — same check semantics
systemd + Quadlet	every container above, as a unit. Podman. No Kubernetes.

The eight seams

If a new repo doesn't plug into exactly one of these, it doesn't go in.

AgentRuntime	Hermes or BeeAI or OpenJarvis – one at a time
ModelRouter	litellm → llama.cpp / vLLM
Executor	android · browser · linux · api · workflow · human
Capability	your names and schemas; implementations are executors
Verifier	Playwright assertions, pixelmatch/odiff, ffprobe, qpdf
Policy	Constitution → SpiceDB → Zitadel
Package	VAPP manifest → wasmtime / Podman / Flatpak
Delivery	cosign → Quay/Pulp → Candlepin → bootc

NEVER LEAK The moment a VAPP manifest names Playwright or a resource-id, the executor stops being swappable and you're back to stacked shells. **Capabilities name *what*. Executors decide *how*.**

15

Open decisions

Each of these has stayed open across multiple document generations. Not choosing means building and maintaining two integrations forever — the contracts let you *swap* later, they don't let you run both for free.

DECISION	RECOMMENDATION	WHY
Mender vs bootc	bootc	Your own IBM assessment points the same way — Podman + quadlet, no Kubernetes. Mender's server-side fleet model fights the no-access posture.
Ubuntu Core	drop	Snaps add a second packaging system and Canonical's store in your update path.
Zitadel vs Keycloak	pick one	Backstage in → Keycloak (Red Hat ships the integration). Backstage out → Zitadel is lighter.
SpiceDB vs OpenFGA	pick one	Same Zanzibar model. V2 ledger says SpiceDB — close it there.
Hermes vs BeeAI	pick one	BeeAI is pre-1.0 with an IBM "may not maintain" notice. Your runtime contract makes either swappable — running both makes neither.
NATS vs Kafka	NATS on box	Kafka only if cloud volume forces it. Never both on a node.
TUF alongside cosign	only if needed	TUF earns its place for key rotation and role delegation cosign doesn't give. Real yes/no, not a both.
Temporal vs Restate	Temporal	Your own ledger already says evaluate, not run alongside.
Payments: own the rail?	own it for bookings you create	Or take zero transaction revenue and price the subscription higher. Don't try to invoice for money you never touch.

DECISION	RECOMMENDATION	WHY
Open core vs Red Hat model	Red Hat model	Nothing withheld. Open core creates a two-tier product and undercuts the credibility that's your strongest asset. Your moat isn't in a repository.

Two collisions worth naming because they're wrapper-on-wrapper, not choices:

- **ollama wraps llama.cpp.** `model-router` → `litellm` → `ollama` → `llama.cpp` is two redundant hops. Ollama is a dev tool; don't ship it on the box.
- **mobile-mcp wraps uiautomator2/adb.** It's an MCP projection of your executor, not a second executor. Maestro, trailblaze and appium-inspector are factory-only.

16

IP, credit and priority

Positioning and recordkeeping only. Scope, strength and timing of filings are counsel's call, not covered here.

Credit and protection are different goals with opposite tactics

Protection wants quiet until filings are locked. Credit wants loud, early and dated. You get both by splitting what from how — the same split as the stealth question, pointed at attribution instead of competition.

Publish the what

"A business's own phone, their own SIM, their own logins, operating the real apps they already use — and it shows you a screenshot proving it worked."

Keep the how quiet: the accessibility service architecture, the fingerprint gate, the repair loop, the flow format, the verification-on-a-different-path rule. Those are what take someone from "interesting" to "I can build that."

Dated priority — mechanisms

- **Cryptographic timestamping.** Hash the document, anchor the hash publicly. OpenTimestamps (free, Bitcoin-anchored) or rekor — already in your stack. Produces "this exact file existed on this date," unforgeable, un-backdateable. Ten minutes, costs nothing.
- **Signed git commits and tags.** A dated technical record in a repo you already have.
- **The videos.** Dated, public, you on camera — which is the plan anyway.
- **Third-party coverage.** A journalist writing about it on a date is evidence you didn't create.
- **The 2013 email chain.** Preserve properly — original headers, don't forward it around, archive somewhere with integrity.

COUNSEL Whether and when to publish detail interacts with filings in ways that vary by jurisdiction and timing. The recordkeeping above is a documentation question; the publishing decision isn't.

Demonstrations date better than descriptions

A written description of an idea is easy for someone to claim they had independently. A **dated video of it working on a real business account is not** — the artifact is too specific. Another argument for the thirty-second unbroken take being the first thing you make: it's marketing, proof of concept, and dated public record at once.

The honest perspective

Credit doesn't go to whoever said it first. It goes to whoever shipped it and got customers. Xerox PARC invented the GUI; Apple got the credit. Sun said "the network is the computer" a decade before anyone cared.

If someone announces phone-based remote control next year and you have 200 businesses running it plus a dated video from this month, you win the attribution fight without arguing it. With a blog post and no customers, you don't — regardless of who posted first.

CONCLUSION The strongest protection for credit is the same thing that's strongest for everything else: ship it.

Brand-mark exposure

Apple acquired NeXT in 1997 including the marks, and Paul Rand's cube is still owned IP. The cost of a dispute isn't legal fees — it's **sunk brand equity**: boxes badged, packaging printed, chamber deals signed under the name, a filing that gets opposed and blocks your own registration. And it arrives at whatever moment is worst — mid-raise, mid-production, mid-season.

Keep the brand cheap to change until you know. Run the homage where it's reversible — website, story, deck, videos. Hold the permanent, expensive surfaces — hardware badging, packaging, the filing — until counsel gives a read. Same provocation, none of the switching cost.

17

Founder assets

The things that are hard to acquire and easy to underuse.

DISTRIBUTION

Twenty years of sales

Car sales, door to door, trained reps out of rental operations. Small-business software dies on CAC because you can't afford a salesperson for a \$200/mo customer and they won't self-serve. A founder who can actually sell this buyer is the thing most teams in this category can't buy at any price.

PROOF

The 2013 email chain

Dated evidence you had the thesis before it was obvious. Converts "guy with an idea" into "guy who's been right and waiting." Deck opener: the email on screen with the date highlighted, then scroll into what you built.

NARRATIVE

The denial letters

Gatekeepers who said no, plus the MIT rejection. Framed on the wall is a photograph that ends up in an article. Free, and it can't be manufactured.

CHANNEL

On camera

Most people who can build this won't get in front of a lens; most people who make content aren't building anything. Being both is a genuine niche, and it's how a company with no ad budget gets distribution.

NETWORK

Local operators

30A and Destin relationships, and a path to a multi-unit franchisor. Money with no portfolio conflict, who are customers and references as well as capital, and who will never tell you to go partner with Toast.

METHOD

Playing dumb

Asking someone to explain what you already understand tells you what they know, how they think, and whether they're honest about their limits. People reveal more teaching than defending. Most useful tool you'll have interviewing CTOs.

How to position it

Be careful with "I know more than tech people." A strong engineer finds the gaps in ten minutes — everyone has them — and if you've claimed otherwise you've spent credibility you didn't need to spend. The stronger line, and the true one:

"I'm not an engineer. I know this business, this customer, and how every piece fits together better than anyone. I need someone who knows the systems better than I do."

Same rule with the prediction ledger: **specificity beats magnitude**. Three dated, checkable predictions land harder than "I predicted 90% of it." You have receipts — receipts beat claims, and you don't need the claim once you show the receipt.

Focus, resolved

"Focus on one thing" and "the pieces only work together" are both right — they're about different things.

- **The architecture has to be broad.** Email ingestion without execution is a parser. Execution without verification is a liability. That's design, not scope creep.
- **The go-to-market has to be narrow.** Amazon sold books. AWS launched with S3. Both had the whole thing in mind and neither led with it.

Build the platform, sell one sentence. You keep the entire vision; focus only has to happen in the sales call, which is the one place you're already better than anyone.

SELF-KNOWLEDGE The instinct to avoid the comfortable six figures is real and rare. But the risk doesn't go away when this works — it gets bigger, right after the first good quarter. What protects against it isn't discipline, it's obligation: employees whose rent depends on you, agencies with books of business, businesses that would be stranded. Build those in early and they'll hold you to it when you don't feel like being held.

18

Risks

App anti-automation detection

TEST WEEK ONE Some apps — banking especially, via Play Integrity — refuse to run with an active accessibility service. This is the only item that could be a hard wall rather than a hard problem, and it costs a weekend and one phone. It determines your addressable market.

Flow brittleness

REPLAY, THEN VERIFY Pure vision agents land 60–85% on novel multi-step tasks — fine for hours, unacceptable for invoices. Recorded deterministic paths plus mandatory end-state verification gets to ~98%, and a failed verification is a fleet signal rather than a customer complaint.

Scope

SHIP P2 BEFORE P4 Immutable OS, model orchestrator, GUI agent, data plane, marketplace, directory, hardware — each is a company. Only one sells. Software on the customer's own hardware buys revenue and evidence before you commit to inventory you can't return. And the documents have started auditing each other — 643 repos catalogued, no customer in any of them.

Email template churn

SAME MACHINE FareHarbor redesigns its confirmation and your parser breaks. Fingerprint the format, fail loudly rather than parse wrong, re-map once, push to everyone.

Phone as a wear item

CERTIFY, DON'T SHIP A random \$40 Android reintroduces the hardware matrix. Publish a supported device list — Moto G, Nokia/HMD, Samsung A-series, refurb Pixel. **Avoid Xiaomi MIUI, Oppo ColorOS, Vivo** — aggressive battery management silently kills the accessibility service, the single most likely failure mode in the design. And duty-cycle USB power in the cradle: charge to 80%, cut, drift to 60%. A phone pinned at 100% swells.

Overselling

NEVER CLAIM SMARTER Local models trail frontier models and customers notice in a week. Same mistake with the free app store — self-hosted alternatives are worse than the paid thing. Frame both as generosity, not replacement. Sell private, flat-fee, offline-capable, yours.

Single-vendor concentration

TWO VERTICALS EARLY If 80% of flows live inside one POS vendor, their roadmap is your roadmap. Same for payment volume — no vertical past ~35%.

Support cost

WHITE LABEL Consumers and small businesses generate 10–50× the tickets of enterprise admins. Agencies owning tier-1 is the structural answer; the approval queue and a one-button Recover are the product answers.

Data protection without a pile

CUSTOMER-HELD BACKUP A business running invoicing on your box will call you when a drive fails. Encrypted backup to a destination *they* choose — their NAS, their bucket — with a key you never hold. Protected, and you still can't read anything.

Fleet blindness

CERT RACK + OPT-IN RING No telemetry means no canary signal. Replace it with your own certification rack and a voluntary early-access ring. Don't pretend it's free.

The test that makes "decentralized" true

If you shut down tomorrow, does the box still work? Local data, local execution, last-known-good flows cached, no phone-home required to function. If yes, the claim holds and nobody else in the category can make it. If no, it's marketing and someone will find out.

Say it in the sales call — *"if I go out of business, your machine keeps working"* — it answers the biggest objection a small business has about buying from a company they've never heard of.

19

What to do next

This week

1. Buy one unlocked mid-range Android and a working SIM. One mini PC, 32GB+.
2. Install the real apps a Destin charter or restaurant uses. Turn on an accessibility service.
See what still runs. That answer determines everything else.
3. scrcpy mirroring, adb sending input. One afternoon.
4. Dump the UI tree as structured text.
5. Pick one flow — business hours in Google Business Profile. Record a human doing it.
6. Replay it. Screenshot the result. Read it back and confirm the hours changed.
7. Film it in one unbroken take, thirty seconds, no edits.

That video is the company

Your first ten customers, your first hire, your fundraiser. Someone says "update my hours" out loud and a real business's real listing changes on a real account, with proof. Nobody has to be told what it means.

Build it before you buy inventory, file a trademark, or design a screen.

Phases

	WHEN	GOAL	DONE WHEN
P0	Wk 1-4	Prove remote control on one flow	You say "update my hours to 9 to 5" and get a screenshot proving it happened.
P1	Wk 5-12	Flow library + approval board	Three businesses run five flows each, unattended, two straight weeks.
P2	Wk 13-20	Ship software, not hardware	Twenty-five businesses paying monthly, zero boxes shipped.
P3	Wk 21-32	The churn machine	A real app redesign detected, re-mapped and pushed fleet-wide inside 24 hours, no tickets.
P4	Wk 33-48	The appliance	100 units pass the certification suite, shipped to existing software customers first.
P5	Year 2	Widen	Model catalog, voice, regulated tier, multi-location fleet SKU where the customer holds the keys.

The weekly loop that is the subscription

Mon	fleet health – which verifications failed over the weekend
Tue-Wed	re-map broken flows, test on the certification rack
Thu	build and sign the flow pack, canary 5%, watch
Fri	staged rollout, changelog written for a restaurant owner
Ongoing	onboard new apps, add models, grow the shared library

If this loop runs, the business works. If it stops for a month, customers find broken flows before you do — and that's the only way this product dies.

Hiring the CTO

- **Not** a cloud/SaaS architect (wrong instincts — they'll rebuild what you're attacking), not an ML researcher (the agent is deliberately dumb), not a big-company CTO (they manage; you need hands on).
- **Yes:** a systems and infrastructure engineer who has **shipped physical product with software on it**. bootc, Yocto, Buildroot, Home Assistant OS. Fluent in Linux, containers, systemd, OTA. Thinks in assembly, not invention.
- **Where:** Home Assistant, Fedora IoT, Universal Blue and bootc contributor communities — they've already solved your OS problem for fun. Locally, Eglin and Hurlburt have a real concentration of embedded Linux engineers.
- **What makes it work:** their job is to say no, and yours is to accept it sometimes. A CTO who can't slow you down is useless; a founder who won't be slowed will churn through three of them.

The one sentence

"Your website shows what's actually available, right now, across every platform you're on — so people can book you direct instead of through the site taking twenty percent."

Every charter, marina, salon and rental operator on the Gulf Coast understands that immediately, and most of them are already angry about the commission. The architecture has to be broad; the sales call has to be narrow. You get to keep the whole vision — focus only has to happen in the one place you're already better than anyone.

20

Code and companions

What exists in the repo

Branch `claude/redhat-model-hardware-sales-iejnvo` on `the-ghost-ai`. Main is untouched.

<code>ghost/flow.py</code>	the script format – selectors not coordinates, <code>{{variables}}</code> , content-hashed identity
<code>ghost/runner.py</code>	fingerprint pre-flight → steps → read-back verify → run record with screenshots
<code>ghost/device.py</code>	adb: read the screen as nodes, tap, type, screenshot
<code>ghost/recorder.py</code>	map a flow by doing the task once
<code>ghost/pack.py</code>	bundle flows as OCI artifacts, cosign-signed, oras push/pull, refuses unverified installs
<code>platform/</code>	assembly map + Podman quadlets for the foundation spine (Postgres, Keycloak, zot, ContextForge)

Honest note on that code: `device.py` and `recorder.py` duplicate `uiautomator2` and Maestro Studio and should be replaced by them. What's worth keeping is the layer Maestro doesn't have – refuse-to-act gating, read-back as proof rather than assertion, the run record, and `pack/sign/push`. That thin layer is the owned contracts; everything under it is borrowed.

Flow format

```

flow: google_business.update_hours
version: 1
app: com.google.android.apps.town.businessmessaging
requires: [day, open, close]

fingerprint:                                # must be true before we touch anything
  - text: "Business hours"

steps:
  - tap: {text: "{{day}}"}
  - clear: {id: open_time}
  - type: {id: open_time, value: "{{open}}"}
  - tap: {text: "Save"}

verify:                                      # must be true after, read off the screen
  - contains: "{{open}}"
```

Verbs: `launch` · `stop` · `tap` · `type` · `clear` · `wait` · `sleep` · `back` · `home` · `scroll` · `assert`. Deliberately small — a flow anyone can read is a flow anyone can fix at 2am when an app redesigns.

Selectors: `text` (exact), `contains`, `id` (resource-id short name), `desc`, `cls`, `clickable`.

Companion documents

- **Build Plan** — phases P0–P5 with done-when gates, the weekly ops loop, economics, risk register, first thirty days.
- **Parts Catalog** — the full component list by layer with use / study / careful / skip verdicts on each, including licenses that matter for shipping hardware.

Working record, revision two — adds the six unsolved problems, IBM/Red Hat findings, modularity rules and the thirteen wires, IP and credit posture, founder assets, and the code reference.

Revision one notes. Phase durations assume a small team and will move; the dependency order should not. Vendor claims, pricing and product details should be re-verified against first-party sources before being used in legal, investment or public marketing contexts.

PLATFORM ARCHITECTURE

A NEXT GENT Build Spec

How to build the foundation once so every app after it takes a week. Data placement, the record, ingestion, capabilities, the app contract, surfaces, the App Store pipeline, isolation, repo layout, and the order to build it in.

01

The one rule

Apps never own data. Apps are views and behaviors over one shared business record.

Get this wrong and you have four SaaS products that need reconciling forever. Get it right and app number seven takes a week.

Everything in this spec exists to enforce that rule mechanically, so it can't be violated by accident at 2am when a feature is due.

02

Where data lives

The question that determines everything else. A tourist scanning a QR code at 9pm on a Saturday cannot depend on a mini PC in a restaurant's back office being reachable — but they also don't need any private data.

Private truth lives on the node. Public projections live on the edge. The node is the only writer.

NODE (box on the counter, or a cloud container)
canonical private state – customers, transactions, credentials,
visits, bookings, observations, the ledger
the ONLY thing that writes

|
| builds a **projection** – publishable fields only,
| signed, versioned, ~kilobytes
↓

EDGE (cloud, read-only, CDN-fronted)
serves: public page · widget · QR target · JSON-LD · MCP endpoint
holds nothing private, can never write back

|
↓
tourists · AI agents · partner directories · chambers

Why this shape and not the alternatives

- **Everything in the cloud** contradicts the entire product thesis and puts you in the breach-liability business.
- **Everything on the box** means a QR menu goes dark when their internet drops, which is a support call per outage and a bad experience for a customer who isn't yours.
- **The split** gives you CDN-speed public surfaces, zero private data in the cloud, and graceful degradation — if the node dies, the edge keeps serving the last projection. Stale, but alive.

PROPERTY The edge is **write-incapable by construction**, not by permission. There is no code path from the edge back into the node. That's what makes "we can't see your data" a structural claim rather than a promise.

What's in a projection

```
projection/v1/{business_id}/{version}.json    (signed)

business    name, hours, location, contact, policies
items       name, category, price, modifiers, dietary,
            rating_avg, rating_count           ← aggregates only
resources   kind, name, available_count       ← counts, never who
reviews     rating, body, item_ref, date      ← no customer identity
availability slots with remaining counts
updated_at, expires_at, signature
```

The exposure rules decide what enters a projection. Nothing else can. A field that isn't in the projection schema physically cannot reach the internet.

03

The node runs anywhere

This is the design decision that makes your free tier possible without a second codebase.

A node is a node whether it sits on their counter or in your cluster.

Same containers, same schema, same capabilities, same everything. Only the location differs.

TIER	NODE LOCATION	INGESTION	EXECUTION
free	your cloud, one container per business	email forward only	none — read and project only
software	their hardware	email + QR + POS	their phone, their browser
ghost	the box you sold them	all sources	all executors, local models, voice

The upgrade is a migration, not a rewrite. A free-tier business that buys a box gets their node shipped to the hardware — same data, same schema, same apps, new location. That path has to exist from day one or the free tier becomes a dead end you'll have to rebuild.

REQUIREMENT Nothing in the node may assume it is on owned hardware. No hardcoded paths, no assumption of a local phone, no assumption of a GPU. Every capability declares what it needs and degrades cleanly when it isn't there.

04

The business record

One schema, shared by every app. Notice which tables multiple apps touch — that overlap is the entire reason the record exists.

```

-- identity and scope
business(id, name, legal_name, vertical, created_at)
location(id, business_id, address, geo, timezone)
member(id, business_id, person_id, role)
node(id, business_id, kind[box|cloud], pubkey, enrolled_at, last_seen)

-- the operating record
resource(id, business_id, kind, name, capacity, attrs)
item(id, business_id, name, category, price_cents, attrs, active)
item_modifier(id, item_id, name, price_delta_cents)
booking(id, business_id, resource_id, channel, external_ref,
        starts_at, ends_at, party_size, status, amount_cents, customer_id)
visit(id, business_id, started_at, ended_at, table_ref, ticket_ref, total_cents)
visit_item(visit_id, item_id, qty, price_cents)
review(id, business_id, visit_id, item_id, rating, body, verified, published_at)

-- private tier, never projected
customer(id, business_id, phone_hash, email_hash, consent, first_seen)
credential_ref(id, business_id, service, vault_ref) -- never the secret

-- provenance: the table you cannot add later
observation(id, business_id, entity_type, entity_id, field, value,
            source, source_ref, observed_at, confidence)
canonical(business_id, entity_type, entity_id, field, value,
          resolved_at, confidence, from_observation_id)

-- action and proof
action(id, business_id, capability, args_hash, requested_by,
       constitution_version, policy_decision, executor,
       started_at, finished_at, status)
evidence(action_id, kind, hash, path, expected, observed)

-- flows
appmap(id, target_app, target_version, capability, version, digest, status)
appmap_run(id, appmap_id, node_id, action_id, result, drift_signature)

-- apps
app(id, package, version, manifest, signature, certified_at)
install(id, business_id, app_id, installed_at, settings)
grant(id, install_id, capability, scope, purpose, expires_at, revoked_at)

```

How canonical works

canonical is a **materialized view**, not a hand-maintained table. A new observation lands, the resolver recomputes that one field, writes the result. Never UPDATE item SET price — always insert an

observation and let resolution decide.

```
resolution policy, per field class:
1  source rank      menu price:  POS > website > Google > Facebook
                        hours:      owner declaration beats everything
2  staleness decay  40 minutes old outranks 6 days old;
                        both decay toward unknown
3  escalation       two high-trust sources disagreeing past a
                        threshold is an owner question, not an
                        automatic resolution
```

CANNOT RETROFIT Provenance is the one thing you cannot add afterward. If you write values directly for six months and then decide you want source tracking, you have six months of data with no origin and no way to reconstruct it. Build the observation table before the first ingestion runs.

One writer per field

FIELD	SOLE WRITER
booking.status	availability app
item.price_cents	menu app (via POS observation)
review.rating	reviews app
visit.*	ingestion, never an app
canonical.*	the resolver, never an app

Two writers on one field is a conflict you will never fully solve. Enforce it in the capability registry, not in a code review.

Ingestion pipeline

SOURCE	→	NORMALIZE	→	OBSERVATION	→	RESOLVE	→	CANONICAL	→	PROJECT
email forward signed biz-a7f3@in... bundle QR scan POS read remote-control read owner text-back partner feed		parser per vendor+format scan event ticket parse screen read declaration iCal / API		insert (never update)		source rank + decay		materialized view		

Email ingestion, concretely

- **Unique inbound address per business** — biz-a7f3@in.yourdomain.com. Never one shared inbox you parse for hints. Routing must be unambiguous from the envelope, and it means you can revoke one business's ingestion without touching anyone else.
- **Parsers are versioned and fingerprinted.** Each parser declares the vendor, the format signature it expects, and what it extracts. A format mismatch means **quarantine and alert**, never a best-effort parse. A wrong booking is worse than a missing one.
- **Raw mail is retained** (encrypted, on the node) until the parse is confirmed, so a fixed parser can reprocess history.
- **Idempotency** on (vendor, external_ref) — the same confirmation forwarded twice must not decrement inventory twice.

ONBOARDING Rank the setup paths by friction: 1 add your address as a second notification email inside the vendor's settings — Toast, Square and FareHarbor all allow it, zero Gmail involvement. 2 a Gmail filter plus a verified forwarding address, two minutes in the UI. 3 an Apps Script, last resort — "run this script" kills conversion at the exact moment you need it to feel like nothing.

Inventory derivation

```

declared capacity (owner)          6 jet skis
- confirmed bookings (all channels, from email)
+ cancellations
- manual decrements (walk-ins, phone)
- safety hold (configurable, covers email lag)
= published availability

reconcile:  read real platform state through the remote control
             on a schedule; compare; flag drift

```

Email lag is 30 seconds to two minutes. Hold a safety slot — show 4 when you have 5 — configurable per business and tightened as you accumulate real lag data.

06

The capability layer

The seam. Apps, agents, the phone assistant, developers and your own code all go through exactly this and nothing else.

A capability is a postcondition plus executors

```

capability: menu.update_price
  args:          { item_id, price_cents }
  postcondition:
    subject:      item{item_id}
    predicate:     price = price_cents
    surfaces:      [website, google, facebook, qr_menu]
    observable_by: [api.read, browser.read, android.read]
  risk:          high
  writes:         [item.price_cents]
  idempotency:    (item_id, price_cents)
  compensation:   roll_back_applied_surfaces
  settle_window:  120s

```

Defining capabilities by **what must become true** rather than by a sequence of steps is what makes executors interchangeable by construction, and it lets verification be derived from the contract instead of written per-executor.

Every call follows the same path

```
caller (app | agent | voice | SMS | developer | MCP)
  ↓ ActionSpec    capability, args, requested_by, purpose
  ↓ constitution  bind the version in force right now
  ↓ policy        ALLOW · ALLOW_WITH_CONSTRAINTS · REQUIRE_APPROVAL · DENY
  ↓ credential    scoped token, never the raw secret
  ↓ router        score executors: login state, health,
                  determinism, reliability, cost, latency, risk
  ↓ execute       api | browser | android | linux | workflow | human
  ↓ verify        read back on a DIFFERENT path than wrote
  ↓ evidence      screenshot, hash, observed vs expected
  ↓ ledger        append-only, immutable
```

Four possible results, not two: VERIFIED, UNVERIFIABLE_YET (a schedulable state, not an error — propagation delay is normal), PARTIAL, CONTRADICTED.

The starting capability set

read	business.get	item.list	review.aggregate_by_item
	resource.availability	booking.list	visit.list
	customer.cohort_count	← count only, never rows	
write	business.update_hours	item.update_price	item.mark_unavailable
	booking.create	booking.cancel	review.publish
	offer.send_to_cohort	← returns a count, never a list	
surface	page.publish	widget.render	qr.generate
	notify.send	projection.build	
system	appmap.run	appmap.report_drift	ledger.append

RULE customers.list does not exist and never will. If a capability's return type is rows of personal data, the whole consent architecture is already lost. Counts, aggregates and actions — never records.

The app contract

Yours and a third-party developer's declare exactly the same thing. That symmetry is what makes it an ecosystem instead of a codebase with plugins.

```
app: restaurant.menu
version: 1.2.0
vertical: restaurant
display: { name: "QR Menu", icon: menu, category: storefront }

reads:
  - item.list
  - review.aggregate_by_item
  - business.get

writes:
  - visit.record_scan

surfaces:
  public_page: { template: menu, blocks: [header, sections, ratings] }
  qr:          { target: public_page, per: table }
  widget:      { sizes: [full, compact] }
  agent_api:   { schema: [Menu, MenuItem, Offer, AggregateRating] }
  voice:       { intents: [whats_on_menu, item_price, is_available] }

settings:
  show_ratings:    { type: bool, default: true }
  currency:        { type: enum, values: [USD], default: USD }
  sections:        { type: list, of: string }
  hide_unavailable: { type: bool, default: true }

requires:
  ingest: [pos]          # optional – degrades without it
  apps:   []

events:
  emits: [menu.scanned, menu.item_viewed]
  listens: [item.price_changed, item.marked_unavailable]

data:
  owns:   []              # apps own nothing
  scopes: [business, item, review, visit]
```

Six rules the manifest enforces

- 1. **Apps never touch the database.** They call capabilities. That seam is what lets you swap storage, add caching, enforce policy and write the ledger without touching app code.
- 2. **Apps own no tables.** `data.owns` is always empty. If an app needs to persist something private to itself, it gets a namespaced key-value scope — never a table in the record.
- 3. **Surfaces are declared, not built.** The app says which surfaces it uses and what blocks to render; the platform renders them. Otherwise you've built seven websites.
- 4. **Settings are declared, not coded.** The platform renders the owner UI from the schema. Otherwise your owner UI becomes seven config screens.
- 5. **Dependencies are explicit and degrade.** `requires.ingest: [pos]` means the app works better with POS and still works without it. Hard dependencies block install; soft ones show a banner.
- 6. **Events are declared both ways.** What it emits and what it listens for, so the platform can wire it without the app knowing who else exists.

08

Surfaces

Build six once. Every app gets all six free, forever.

SURFACE	RUNS ON	WHAT THE APP DECLARES
public_page	edge	template + ordered blocks
widget	edge (iframe / script)	sizes and which blocks
qr	edge target, node-generated	target surface, granularity (per table, per boat, per room)
sms	node, via the SIM	intents it answers, messages it sends
voice	node	intent names and slots
agent_api	edge	JSON-LD types + MCP tool schema

Block rendering

An app declares blocks. The platform owns the templates, the theme, responsiveness, dark mode, SEO and accessibility. An app cannot ship its own CSS or its own page.

```
blocks: [header, hours, sections, ratings, cta_book, reviews, map]
      ↓
platform renders into: public page · widget · JSON-LD · voice response
```

This is why app #7 costs a week. It writes a manifest and some capability calls; it never writes a page, a stylesheet, a settings screen, a QR generator or an SEO tag.

The agent surface is not optional

Every public page emits Schema.org JSON-LD, an MCP endpoint, llms.txt, and /.well-known/agent-card.json — from day one, before anyone asks for it. A restaurant menu as a PDF is invisible to AI. Being the source AI assistants query is an infrastructure position you only get by being there first.

```
public  JSON-LD + llms.txt      free, anonymous, scrapeable
      ↓
identified  MCP / A2A          live availability, item ratings,
                               booking – requires an identified client

→ you get presence from the first, telemetry and the relationship from the second
```

Events

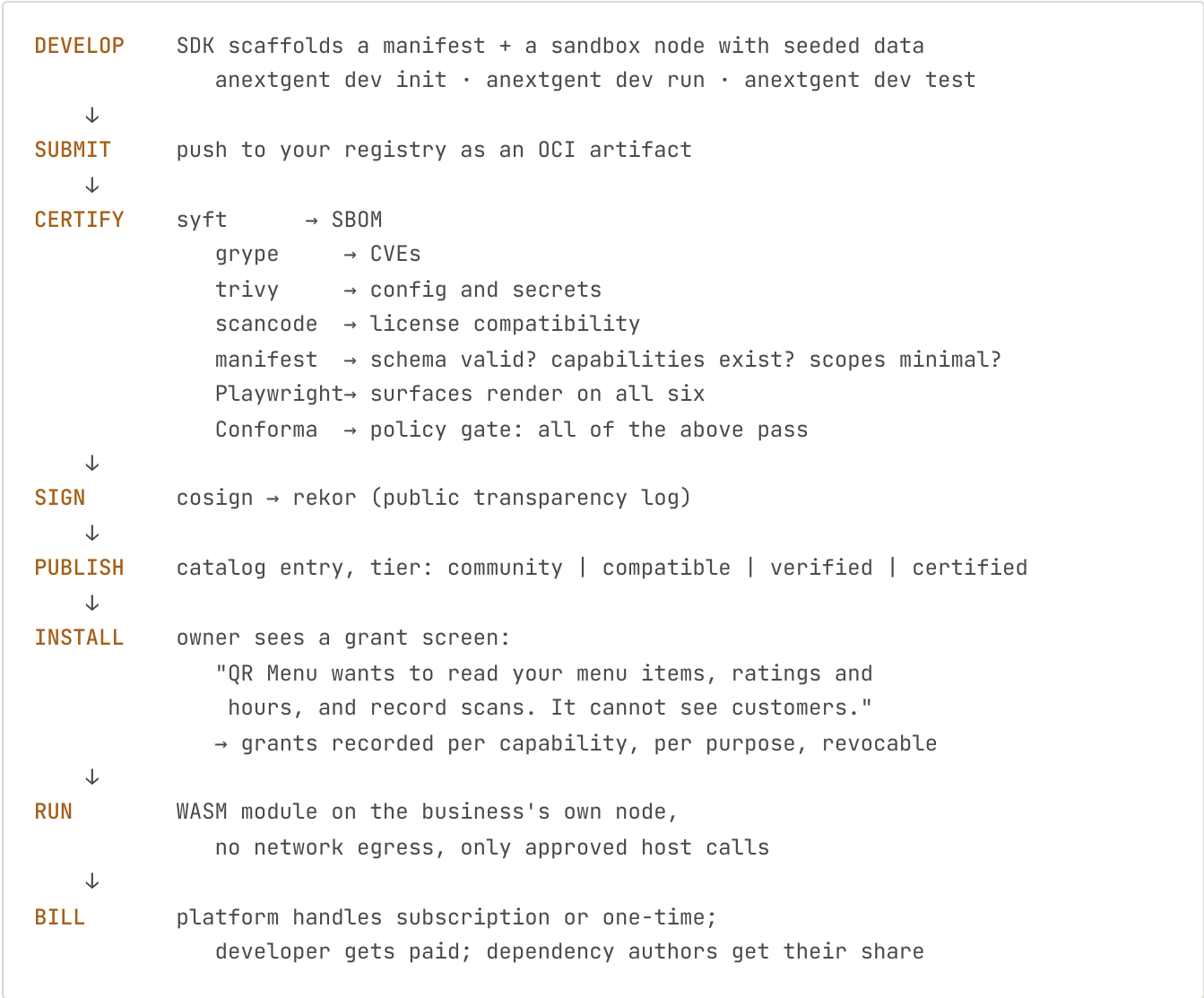
```
ingestion → emits booking.created, item.price_changed, visit.completed
resolver  → emits canonical.changed
apps      → emit  menu.scanned, review.published, offer.sent
executors → emit  action.verified, appmap.drifted
schedule  → emits cron.daily, cron.hourly
```

↓ NATS on the node ↓

```
apps listen      projection rebuild      rulebooks (Event-Driven Ansible)
                                     → re-map drifted flow
                                     → alert the owner
                                     → route to fallback executor
```

DISCIPLINE Events carry **identifiers, not payloads**. `booking.created{booking_id}`, never the booking object. The listener calls a capability to read what it's allowed to read. Payload-carrying events leak data past the policy layer and you'll never find all the places it happened.

The App Store pipeline



Certification tiers

community	Manifest valid, scans pass. No guarantee.
compatible	Automated tests pass on current platform version.
verified	Reviewed and tested by you.
certified	Specific versions, specific hardware, support commitment.

That ladder is a Red Hat innovation and it's how you monetize trust around open source later. Build the field now even if only two tiers are live.

What makes apps compound instead of accumulate

An app that only *consumes* capabilities adds one app. An app that *publishes* one makes every future app easier. The registry knows exactly which app called which — so pay dependency authors from the apps built on them, publish the formula, and publish the payouts. Nobody else can measure this, because nobody else has a registry in the middle.

11

Isolation and tenancy

```
one business  =  one node  =  one database  =  one key

no tenant_id column anywhere in the record
no cross-business query is possible, because there is no
cross-business connection to make
```

Tenant-column multi-tenancy is cheaper to build and leaks eventually — one missing WHERE business_id = and you've had a breach. Node-per-business makes the leak impossible rather than unlikely, and it's what your whole posture depends on.

Inside a node

RUNS AS	ISOLATION	WHY
platform services	Podman + Quadlet	trusted, your code
first-party apps	WASM (Extism / wasmtime)	same rules as everyone else — no privileged path
third-party apps	WASM, no network egress	physically cannot phone home; only approved host functions
untrusted / heavy	gVisor or Firecracker	kernel-level boundary when needed

CRITICAL Your own apps run under the same restrictions as third-party ones. The moment first-party code gets a privileged path to the database, the platform stops being a platform and developers will find out. It also means you discover the SDK's gaps by using it.

Return shaping — the leak nobody plans for

A developer with enough narrow questions reconstructs the dataset. "How many ordered scampi," filtered a thousand ways, rebuilds the customer list one count at a time, and every individual query looks harmless.

- **Minimum cohort size** — results under N (10–25) return nothing
- **Query budget per grant** — not just rate, total questions against a dataset per period
- **Aggregates only for analytics scopes** — action scopes act, they don't report
- **Enumeration detection** in the ledger, flagged to the business

Repo layout

```

anextgent/
  contracts/      ← the owned layer. changes here are versioned events.
    capability/   capability definitions, postconditions, risk
    record/       schema + migrations
    manifest/     app manifest JSON Schema
    actionspec/   the request object
    projection/   what may be published
    constitution/ authority schema

  node/          runs on a box OR in your cloud – identical
    ingest/       email · qr · pos · remote-read · owner-text
      parsers/    per vendor, versioned, fingerprinted
    resolve/      observation → canonical
    registry/     capability registry + router
    policy/       constitution + policy engine + credential broker
    execute/      android · browser · linux · api · workflow · human
    verify/       per-capability postcondition checks
    ledger/       append-only
    project/      builds signed public projections
    host/         WASM app host + host functions
    surface/      sms · voice

  edge/          cloud, read-only, no private data
    page/ widget/ jsonld/ mcp/ qr/

  control/       cloud
    identity/ entitlement/ fleet/ appstore/ billing/ metering/

  apps/          first-party – no privileged access
    menu/ availability/ reviews/ booking/ songrequest/

  sdk/           what a developer installs
  factory/       flow mapping, cert pipeline, parser authoring
  os/            bootc image, quadlets, provisioning

```

DISCIPLINE contracts/ is the only directory that is truly yours forever. Everything else is an implementation of something in it. A change there is a versioned event with a migration path — never an edit.

13

Technology per component

COMPONENT	USE	WHY
node runtime	Podman + Quadlet	systemd units, no daemon, no Kubernetes, identical on box and cloud
node OS	bootc + ostree + greenboot	one signed image, atomic swap, automatic rollback
node database	Postgres	one per node. Same engine cloud and box — don't run SQLite in one and Postgres in the other
embeddings	pgvector	already there
events	NATS	lightweight, runs on a node, no ZooKeeper
app sandbox	Extism + wasmtime	host functions are the only surface; no egress
durable jobs	Temporal	survives crash, reboot, a sleeping phone; resumes exactly
rulebooks	Event-Driven Ansible	sources → rules → actions, already written
android	ADB + uiautomator2 + scrcpy	uiautomator2 is the only runtime executor; Maestro stays in the factory
browser	Playwright	also your verifier for web postconditions
parsing	Tika + Docling + langextract	don't write a parser per file type
inference	llama.cpp + llama-swap, litellm in front	hot-swap, process isolation, one endpoint
guardrail	Granite-Guardian on vLLM	called by the policy engine, no Kubernetes

COMPONENT	USE	WHY
voice	Wyoming + openWakeWord + silero-vad + faster-whisper + Kokoro	each stage swappable; Piper is archived
intents	hassil	deterministic routing before spending a model call
identity	Zitadel	OIDC, passkeys — don't build auth
authorization	SpiceDB	owner → business, agent → VAPP, device → workspace
secrets	OpenBao	behind the credential broker; apps get refs, never secrets
registry	Quay (cloud) + zot (node)	OS images, apps, flow packs — one warehouse
signing	cosign + rekor	and put constitution versions and appmap promotions in the log, not just images
entitlement	Candlepin	what did they buy, what may this node install
fleet	Foreman / flightctl	evaluate flightctl before writing fleet code
edge	Next.js on a CDN	static projections, near-zero cost
owner UI + phone	Flutter	BSD-3, one codebase for wall screen and mobile, no Qt licensing
wall screen	cage (Wayland kiosk)	one app fullscreen, no desktop to fall out of
page builder	Puck	owner-editable public pages over the same blocks
TV nav	Norigin Spatial Navigation	D-pad and remote

COMPONENT	USE	WHY
payments	Stripe Connect	destination charges; decide dispute liability deliberately
billing / metering	Lago	entitlement is "may they," metering is "how much"
networking	headscale / WireGuard	phone app → node, point to point, nothing readable in the middle
backup	restic + Litestream	to a destination they choose; you never hold a key
export	xlsx writer from Postgres	the artifact that makes "you own it" legible

14

Build order

Every milestone produces something demonstrable. Build out of sequence and you rework each step.

Phase A — the record and the seam · weeks 1–5

1. contracts/record — schema for **one vertical**, plus observation and canonical
2. node/resolve — source rank, staleness decay, escalation
3. contracts/capability + node/registry — the first 15 capabilities
4. node/ledger — append-only from the very first write
5. node/policy — constitution, the four-state ladder, credential broker

DEMO A capability call that resolves policy, executes nothing, and writes a ledger entry. Boring, and everything depends on it.

Phase B — ingestion and projection · weeks 6–10

6. `node/ingest/email` — unique inbound address, parser framework, quarantine, idempotency
7. Three real parsers — one POS, one booking platform, one review notification
8. `node/project` — signed projection bundles
9. `edge/` — page renderer, widget, QR target, JSON-LD, MCP, `llms.txt`, agent card

DEMO A business forwards booking emails and a live public page appears with real availability. That alone is the free tier — no box, no phone, no install.

Phase C — the app host · weeks 11–14

10. `contracts/manifest` + `node/host` — WASM host, host functions, capability-scoped
11. Block rendering — the app declares, the platform renders
12. Settings schema → owner UI generation
13. Install, grant, revoke
14. `sdk/` — init, run, test against a seeded sandbox node

DEMO A hello-world app with fifteen lines of manifest gets a public page, a widget, a QR code, an agent endpoint and a settings screen — none of which it wrote.

Phase D — the first apps · weeks 15–22

In dependency order. Each needs the one before it.

15. **QR Menu** — no ingestion required, immediate value, seeds item
16. **Reviews** — needs visits, which need scans plus POS
17. **Availability** — needs email ingestion. Most valuable, most complex
18. **Booking** — needs availability

Phase E — execution · weeks 23–30

19. `node/execute/android` — adb, uiautomator2, scrcpy
20. AppMap format, fingerprint gate, refuse-to-act

- 21. node/verify — read back on a different path
- 22. Fleet drift aggregation → rulebook → re-map → sandbox → canary → promote
- 23. SMS and voice surfaces

Phase F — the store and the box · weeks 31+

- 24. Certification pipeline, signing, transparency log
- 25. Catalog, tiers, install flow, billing, dependency revenue share
- 26. bootc image, quadlets, factory provisioning, cert rack
- 27. Node migration — cloud tier to owned hardware

15

The first four apps

APP	READS	WRITES	SURFACES	NEEDS
menu	item.list · review.aggregate · _by_item · business.get	visit.record_scan	page · qr · widget · agent · voice	—
reviews	visit.list · item.list	review.publish	sms · page · agent	menu, POS or QR
availability	resource.availabi · lity · booking.list	booking.create · booking.cancel	widget · page · agent · voice	email ingest
booking	resource.availabi · lity · business.get	booking.create · notify.send	page · widget · sms · voice · agent	availability, payments

Look at the overlaps. Menu and reviews both touch item. Availability and booking both touch booking. Reviews needs visits that the menu app creates. If each owned its own copy you'd have four sources of truth for the same fields and permanent reconciliation work — which is exactly what four separate SaaS products are.

Song requests is the proof the platform is real

QR on a stage → page → request → payment confirmation parsed → identity matched → artist notified. **Zero new architecture.** Five capabilities you already built. If it takes a week, the foundation works. If it takes a month, something in Phases A–C is wrong and you should fix that before app number six.

What breaks if you skip

SKIP THIS	WHAT HAPPENS	RECOVERABLE?
observation table	Six months of data with no origin, no confidence, no way to reconstruct. Conflict resolution becomes guesswork forever.	No
one writer per field	Two apps race on booking.status. Intermittent, unreproducible, and it corrupts customer-facing availability.	Painful
capability seam	Apps reach into the database. Policy, ledger and swappable storage all become impossible without rewriting every app.	No
node-per-business	One missing WHERE business_id = is a cross-tenant breach, and your entire posture is gone in one incident.	No
platform-owned surfaces	Seven apps, seven websites, seven settings screens. App #7 costs a month instead of a week.	Expensive
projection boundary	Private fields leak to the edge. The one mistake that ends the company.	No
ledger from write #1	No audit history for the period you most need it, and Proof Ledger becomes a claim you can't back.	No
first-party apps sandboxed	Privileged shortcuts accumulate, the SDK's gaps stay invisible, and developers find out.	Painful
node location-agnostic	Free tier becomes a dead end with no upgrade path, and you rebuild it as a separate product.	Expensive

SKIP THIS	WHAT HAPPENS	RECOVERABLE?
idempotency on ingest	A forwarded confirmation decrements inventory twice. You oversell. The customer loses trust in the number.	Painful

Seven of those ten are **not recoverable** — they'd require rebuilding the foundation with live customers on it. They're all in Phase A and B, they're all cheap to do first, and every one of them is boring.

STILL TRUE

None of Phase E matters if an accessibility service can't run on the apps a Destin charter operator actually uses. One phone, one afternoon, before week 23.

Build spec, revision one. Phase durations assume a small team and will move; the dependency order between phases should not. Companion documents: the Playbook (strategy and economics), the Build Plan (phases and go-to-market), the Parts Catalog (component list with verdicts).

BUILD PLAN • SMALL BUSINESS AI APPLIANCE

Remote Control Build Plan

A box that runs a small business by operating the apps that business already uses — on a real Android handset with a real SIM — and a subscription that exists because those apps never stop changing.

BUYER	PRICE	SOLD	STACK
Small business owner	~\$200/mo + hardware	Subscription to upkeep	Fully open source
FIRST SHIP	Software, not hardware		

01 — THE THESIS

You are selling churn absorption

Red Hat's subscription is not access to Linux. It is a promise to absorb upstream churn — kernel changes, CVEs, package breakage — so an enterprise never has to. Customers pay every year because upstream never stops moving.

Your upstream is different and far more volatile: **the user interfaces of the apps a small business runs on**. Toast redesigns the hours screen. Square moves a button. Google Business Profile ships a new layout. Every do-it-yourself automation in America breaks silently, and the owner finds out from an angry customer standing at a locked door.

Yours breaks too. The difference is that it breaks for about six hours.

01 A vendor ships a UI change to their app.	02 Post-action verification fails across the fleet. You know before any customer does.	03 You re-map that one flow. Once.
04 The fix is signed and pushed to every customer overnight.	05 Nobody noticed anything happened.	

That is the entire product and the entire business model in one line. It also explains the economics: the flow library is *shared*. Mapping Toast's hours screen costs you the same whether you have four customers or four hundred. **Your marginal cost for the 401st Toast customer is close to zero, and your value to them is identical.**

It also means this can never be a one-time sale. The churn doesn't stop, so neither does the subscription — and unlike a feature subscription, the customer feels it the week you stop paying attention.

02 — PRODUCT

Four parts, one appliance

Everything below is assembled from open source. Nothing here requires a vendor partnership, an API key, or anyone's permission.

THE HAND A real Android phone Customer's own SIM, customer's own logins, done through each app's normal screens. A physical handset with a real IMEI defeats the emulator detection that blocks every container-based approach —	THE BRAIN The orchestrator box Local models decide what to do and read back what happened. Cloud models are optional and bring-your-own-key. The machine holds the memory, the flow library, and the business's context. Nothing leaves unless the owner sends it.
--	--

and SMS two-factor simply works, because it is a phone.

THE FACE

The wall screen

A ten-foot tile layout everyone already knows from a streaming box. Eight tiles, no menus, no filesystem. Glanceable from across the room, driven by a remote, a phone, or voice.

THE BRAKE

The approval board

Three things need your OK. Each with a draft, a dollar amount, and a screenshot of what the agent is about to do. The agent proposes; the owner disposes. This is the trust mechanism and the support-ticket deflector in one.

03 — STACK

What you actually build on

LAYER	CHOICE	WHY THIS ONE
OS base	Debian stable or CentOS Stream, <code>image mode</code> (bootc / OSTree)	One signed artifact per release. Atomic swap, automatic rollback on failed health check, zero migration scripts, identical on every unit.
Boot & integrity	systemd-boot + unified kernel image, Secure Boot with owner-enrollable keys	Locked by default, unlockable by whoever bought it. Sidesteps GRUB2 entirely.
Update transport	OCI registry + <code>cosign</code> signatures + public transparency log	Pull-only. You publish; devices choose. No pipe into anyone's machine means nothing to compel, steal, or target.
Phone mirror	<code>scrcpy</code> over USB	Already solved. Low-latency Android screen on the Linux box with passthrough input. Do not rebuild this.
Phone control	Custom Android Accessibility Service + ADB transport	Gives you the semantic UI tree — real buttons with real IDs — not pixel guesswork. The sanctioned mechanism for software acting on a user's behalf.
Flow engine	Record → replay → verify, vision model as fallback	Owner performs the task once; you capture the deterministic path. Replay is near-perfect; vision only covers what moved. Gets <code>more</code> reliable with use.
Inference	<code>llama.cpp</code> / vLLM, GGUF weights, LoRA hot-swap	One strong base resident in RAM plus small adapters. Specialization swaps in ~200 ms instead of reloading a 5 GB model.

LAYER	CHOICE	WHY THIS ONE
Silicon	AMD Strix Halo (Ryzen AI Max+), 64–128 GB unified	Only affordable way to give a GPU that much addressable memory — and <code>amdgpu</code> is in-tree, which is what keeps the single-image architecture clean.
Tool protocol	MCP	The emerging standard for agent tool calls. Adopt it rather than inventing a plugin format nobody else speaks.
Human apps	Flatpak, curated remote	Sandboxed, versioned separately from the OS, and the free Linux catalog costs you nothing to include.
Local data	SQLite + <code>sqlite-vec</code> , LUKS full-disk, backup to a destination the customer picks	They get protection from a dead drive; you never hold a key and never hold their books.
Voice & SMS	The SIM in the phone	The business's own number. No third-party telephony vendor, no relay, no per-message cost, no unfamiliar caller ID.

04 — SEQUENCE

Build order

Numbered because it is a real dependency chain, not a menu. Each phase exists to retire one specific risk before you spend money on the next.

P0

Prove remote control on one flow

Weeks 1–4

One laptop, one Android phone, one app, one task. Resist every urge to generalize.

- scrcpy mirroring, ADB transport, accessibility service dumping the UI tree
- Record a human changing business hours in Google Business Profile
- Replay it. Screenshot the end state and read it back to confirm

DONE WHEN You say "update my hours to 9 to 5" and get back a screenshot proving it happened.

P1

Weeks 5-12

Flow library and approval board

Turn one trick into a system. This is where the product becomes real.

- Record-and-replay engine with per-app flow definitions
- Verification on every action — never report success without visual proof
- Approval queue UI, first version, on a plain monitor
- Ten flows across three real businesses in your own city

DONE WHEN Three businesses run five flows each, unattended, for two straight weeks.

P2

Weeks 13-20

Ship software, not hardware

Sell it before you manufacture anything. Customers bring their own mini PC and phone; you charge the subscription. This retires demand risk for the price of a Stripe account instead of the price of a container of inventory.

- Installer, licensing, billing, onboarding that a non-technical owner can finish alone
- Twenty to forty flows across two verticals — pick restaurants and one service trade

DONE WHEN Twenty-five businesses are paying monthly and you have not shipped a single box.

P3

Weeks 21–32

Build the churn machine

The actual company. Everything before this was a demo; this is the thing customers renew for.

- Fleet-wide breakage detection from verification failures
- One-fix-maps-to-all pipeline with signed flow packs
- Weekly release train: canary 5%, staged rollout, human-readable changelog
- A certification rack running every supported app version

DONE WHEN A real app redesign is detected, re-mapped, and pushed fleet-wide inside 24 hours — and no customer opens a ticket.

P4

Weeks 33–48

The appliance

Only now does hardware make sense, because you know exactly what people use it for.

- bootc image pipeline, signing in an HSM, transparency log, staged rollout controller
- Strix Halo box with a powered phone cradle and charge limiting
- Ten-foot tile UI in Wayland kiosk mode
- ODM order, factory provisioning, certification suite in CI

DONE WHEN One hundred units pass the full certification suite and ship to existing software customers first.

P5

Year 2

Widen

- Curated local model catalog — the one-click Hugging Face, quantized and tested on your exact box
- Voice and inbound call handling on the business line
- Regulated tier for clinics and firms: retention controls, compliance logging
- Multi-location fleet SKU where the customer holds the management keys

05 — OPERATIONS

The week that is the subscription

If this loop runs, the business works. If it stops for a month, customers start finding broken flows before you do — and that is the only way this product dies.

MON Fleet health. Which verifications failed over the weekend, and on which app versions.	TUE - WED Re-map broken flows. Test against the certification rack, not a developer's phone.	THU Build and sign the flow pack. Canary to 5% of the fleet. Watch.
FRI Staged rollout to everyone. Changelog written for a restaurant owner, not an engineer.	ONGOING Onboard new apps customers ask for. Add models to the catalog. Grow the library that every customer shares.	

06 — ECONOMICS

What it costs and what it returns

LINE	FIGURE	NOTE
Subscription	\$200 / mo	Flat. Not metered — a decisive advantage over per-token cloud AI for an owner who hates surprise bills.
Software-only tier	\$150 / mo	Customer's own hardware. Your P2 product and your lowest-risk funnel.
Appliance, 64 GB	\$2,495	BOM roughly \$1,100–1,500 at low volume. Hardware is the funnel, not the profit.
Appliance, 128 GB	\$3,495	For businesses running larger local models or several at once.
Phone	\$200–400	Include it. Controlling the handset model is what keeps your flow library testable.
100 customers	\$240K ARR	Plus hardware. Roughly one person's full-time churn loop supports this.
500 customers	\$1.2M ARR	Same flow library. The loop grows sublinearly — this is the leverage.
Replaces	\$800–2,500 / mo	A part-time assistant, or the stack of disconnected subscriptions the owner is already paying for.

The number that matters most is not ARR — it is **flows per customer**. A business running two flows will churn. A business running fifteen has moved its operations onto your box and will not leave. Instrument that from day one and treat it as the only retention metric.

07 — RISK REGISTER

What will actually go wrong

App anti-automation detection

TEST WEEK ONE Some apps — banking especially, via Play Integrity — refuse to run with an active accessibility service. This is cheap to check and it defines your addressable market. Run every target app on a real handset before you write a line of the flow engine, and keep a per-app go / no-go list.

Flow brittleness

REPLAY, THEN VERIFY Pure vision agents land somewhere around 60–85% on novel multi-step tasks — fine for hours, unacceptable for invoices. Recorded deterministic paths plus mandatory end-state verification is what takes it to shippable, and a failed verification is a fleet signal, not a customer complaint.

Three hard products at once

SHIP P2 BEFORE P4 Immutable OS, model orchestrator, and GUI agent are each a company. Only the third one sells. Software on the customer's own hardware buys you revenue and evidence before you commit to inventory you cannot return.

Phone as a wear item

CHARGE LIMITING A handset pinned at 100% forever swells. Cap charge in the cradle, schedule reboots, watch for app auto-updates changing UIs overnight, and treat the phone as a serviceable part with a replacement path.

Overselling local model quality

NEVER CLAIM SMARTER Local models trail frontier cloud models and customers will notice within a week. Sell what is actually true and actually unavailable elsewhere: private, flat-fee, offline-capable, and theirs. You already said you are not trying to beat ChatGPT — keep that in the marketing, not just the strategy.

Single-vendor concentration

TWO VERTICALS EARLY If 80% of your flows live inside one POS vendor, that vendor's roadmap is your roadmap. Spread across at least two unrelated trades by P2 so no single app redesign can take down the business.

08 — START HERE

The next thirty days

- 01** Buy one unlocked mid-range Android handset and put a working SIM in it. Buy one mini PC with 32 GB or more.
 - 02** Get scrcpy mirroring the phone and ADB sending input. One afternoon.
 - 03** Write the accessibility service that dumps the UI tree as structured text. This is the foundation of everything.
 - 04** Pick exactly one flow: business hours in Google Business Profile. Record a human doing it.
 - 05** Replay it. Screenshot the result. Read the screenshot back and confirm the hours changed.
 - 06** Film the whole thing in one unbroken take, thirty seconds, no edits.
-

That video is the company

It is your first ten customers, your first hire, and your fundraiser. Someone says "update my hours" out loud and a business's real listing changes on a real account, end to end, with proof. Nobody has to be told what it means.

Build it before you buy a single piece of inventory, register a single trademark, or design a single screen.

Working plan, revision one. Phase durations assume a small team and will move; the dependency order between phases should not. Every component named here is open source and available today.

COMPANION TO THE BUILD PLAN

Remote Control Parts Catalog

Every open-source component the appliance needs, grouped by layer, with a verdict on each. Opinions included — a list of sixty repositories without them is just more work for you.

- USE

Build on this
- STUDY

Read it, don't depend on it
- CAREFUL

Real catch attached
- SKIP

Wrong tool here

THE SPINE

Nine repos carry the whole product

Everything else in this catalog is supporting cast. If you only cloned these nine, you would still have the shape of the machine.

Genymobile/scrcpy

Mirror and control the Android handset.
Solved problem — do not rebuild it.

mobile-dev-inc/maestro

Declarative YAML mobile flows. The shape
your flow library should take.

ggml-org/llama.cpp

Local inference, GGUF weights, LoRA
adapter loading.

mostlygeek/llama-swap

Hot-swaps models behind one OpenAI-
compatible endpoint. Your model manager,
already written.

rhasspy/wyoming

The protocol that wires wake word →
speech → intent → voice into one pipeline.

containers/bootc

The OS as a signed container image. Atomic
swap, automatic rollback.

sigstore/cosign

Sign every image and flow pack; publish to a public transparency log.

modelcontextprotocol

How agents call tools. Adopt the standard instead of inventing a plugin format.

home-assistant/operating-system

Your closest living analogue. Read this one before you write anything.

LAYER 00**Reference builds — read these first**

Four projects have already solved large parts of what you are attempting. Time spent reading them is the cheapest engineering you will ever do.

PROJECT	LICENSE	WHAT IT GIVES YOU	VERDICT
<u>home-assistant/operating-system</u>	Apache-2.0	The single most important repo in this catalog. An immutable appliance OS with A/B boot slots, RAUC over-the-air updates, and a non-technical owner on the other end. Same problem you have, already shipped to millions of devices.	STUDY
<u>home-assistant/core</u>	Apache-2.0	The automation engine, the device abstraction, and the Assist voice pipeline. Also the business template: Nabu Casa sells the hardware and a cloud subscription while the Open Home Foundation holds the project. That is your model with the serial numbers filed off.	STUDY
<u>ubblue-os/bluefin</u> · <u>ubblue-os</u>	Apache-2.0	Universal Blue builds polished, opinionated desktops as bootc images in public CI. The best worked example of the exact build pipeline you need for the OS layer.	USE

PROJECT	LICENSE	WHAT IT GIVES YOU	VERDICT
omacom/omarchy	MIT	Read manual/ for documentation standard, themes/ for how theming becomes a product feature, and install/hardware/ as a list of components to avoid. Do not adopt its mutable-Arch update model.	STUDY

LAYER 01

Immutable OS and updates

PROJECT	LICENSE	ROLE	VERDICT
<u>containers/bootc</u>	Apache-2.0	Your OS pipeline. Boot a machine directly from an OCI container image. Build in CI, sign, push; devices pull and swap atomically.	USE
<u>ostreedev/ostree</u>	LGPL-2.1	The content-addressed filesystem underneath bootc. Gives you file-level deduplicated deltas, so a full OS update downloads like a patch.	USE
<u>fedora-iot/greenboot</u>	LGPL-2.1	The auto-rollback. Runs health checks after boot; a failed check reverts to the previous image before the owner ever sees a problem. This is what makes updates unscary.	USE
<u>rauc/rauc</u>	LGPL-2.1	Classic A/B slot updater — what Home Assistant OS runs. The right choice only if you go Buildroot/Yocto embedded instead of OCI images.	CAREFUL
<u>osbuild/osbuild</u>	Apache-2.0	Turns your bootc image into installable media — ISO, raw disk, the factory flash artifact.	USE

PROJECT	LICENSE	ROLE	VERDICT
systemd-sysupdate	LGPL-2.1	Minimal A/B image updater already in systemd. Worth knowing, but bootc gives you more for the same effort.	STUDY
<u>mendersoftware/mender</u>	Apache-2.0	Full fleet OTA with a management server. Capable, but the server-side fleet model conflicts with your pull-only, no-access posture.	SKIP

LAYER 02

Boot integrity and signing

PROJECT	LICENSE	ROLE	VERDICT
systemd-boot + UKI	LGPL-2.1	Unified kernel images signed as one object. Simpler than GRUB2 and avoids its licensing entanglement entirely.	USE
<u>Foxboron/sbctl</u>	MIT	Secure Boot key management that a human can actually operate. Pair with <code>mokutil</code> so owners can enroll their own keys and unlock the machine.	USE
<u>sigstore/cosign</u>	Apache-2.0	Signs every OS image and flow pack. Devices verify before staging. Keys live in an HSM.	USE
<u>sigstore/rekor</u>	Apache-2.0	The transparency log. Devices refuse anything not publicly logged, which makes a targeted backdoor impossible rather than merely promised. This is the repo that turns your privacy claim into a proof.	USE
<u>project-zot/zot</u>	Apache-2.0	Lightweight OCI-native registry. Simpler to self-host than Harbor and enough for a fleet in the thousands.	USE

PROJECT	LICENSE	ROLE	VERDICT
fwupd/fwupd	LGPL-2.1	Firmware updates through LVFS. Cheap to add and it makes the appliance feel like a real product rather than a PC in a case.	USE
tpm2-software/tpm2-tss	BSD-2	Device identity sealed in the TPM. Backs entitlement without a user account, and disk encryption without a password prompt on a wall-mounted box.	USE

LAYER 03

Android remote control — your core IP

Everything else in this catalog is assembly. This layer is where your actual product gets built, so use these to learn the technique rather than to inherit someone's architecture.

PROJECT	LICENSE	ROLE	VERDICT
Genymobile/scrcpy	Apache-2.0	Mirroring and manual control. Low-latency Android screen on the Linux box with input passthrough. Ships a server-side component you can drive programmatically. Non-negotiable.	USE
AccessibilityService (Android SDK)		You write this yourself and it is the heart of the product. It hands you the semantic UI tree — real buttons with real IDs and text — plus gesture dispatch. Structure beats pixels by a wide margin.	USE
mobile-dev-inc/maestro	Apache-2.0	Read this for the flow format. Declarative YAML UI flows with built-in waiting and assertions — exactly the shape your flow library wants, and it already solved the tedious parts of element matching.	USE
openatx/uiautomator2	MIT	Python control of Android UI. The fastest path to a working prototype in week one, before you write your own service.	USE

PROJECT	LICENSE	ROLE	VERDICT
droidrun/mobilerun	open source	LLM-agnostic agent that inspects UI state, taps, types, and plans multi-step flows via CLI or Python. Closest working reference to your agent loop.	STUDY
sktyou/OpenGUI	open source	Android GUI agent built on standard AccessibilityService APIs, with a local REST API for remote driving. Good architecture reference for the service you will write.	STUDY
OpenBMB/AgentCPM-GUI	Apache-2.0	An 8B on-device GUI agent model that takes phone screenshots and executes tasks. Your vision fallback when the accessibility tree goes blind on a WebView or canvas.	USE
google-research/android-world	Apache-2.0	Your test harness. A benchmark of real Android tasks. Do not guess at your flow reliability — measure it against a standard suite and watch the number move.	USE
appium/appium	Apache-2.0	The mature, heavyweight option. Built for CI test farms, not a live appliance — the ceremony costs more than it returns here.	SKIP

PROJECT	LICENSE	ROLE	VERDICT
waydroid/waydroid	GPL-3.0	Android in a container on the Linux box. Tempting, but it forfeits the real SIM and gets flagged by exactly the POS and banking apps you need. The physical handset is the whole point.	SKIP

Test detection before you build anything on this layer

Some apps refuse to run with an active accessibility service. Install every app in your target vertical on a real handset, turn your service on, and see what still works. One afternoon, one phone, and it tells you what your addressable market actually is.

Local inference and the model catalog

PROJECT	LICENSE	ROLE	VERDICT
ggml-org/llama.cpp	MIT	The engine. GGUF weights, quantization tooling, LoRA adapter loading, an OpenAI-compatible server, and first-class AMD support through ROCm and Vulkan.	USE
mostlygeek/llama-swap	MIT	Your model manager, already written. A single Go binary that sits in front of llama.cpp or vLLM, inspects the requested model, starts the right process, proxies the call, and evicts idle instances. Process isolation means one model crashing cannot take the others down.	USE
vllm-project/vllm	Apache-2.0	Higher throughput and multi-LoRA serving, at the cost of a much heavier runtime. Right for a business box serving several staff at once; overkill for a single owner.	CAREFUL
ollama/ollama	MIT	Easiest developer experience and great for your first two weeks. Hands you less control over quantization and memory than you will eventually want.	CAREFUL

PROJECT	LICENSE	ROLE	VERDICT
huggingface/huggingface_hub	Apache-2.0	The client behind your one-click catalog. You mirror, quantize, test on your own SKU, sign, and serve — the customer never sees a model card or a config file.	USE
ROCm/ROCm	MIT	The AMD compute stack for Strix Halo. In-tree kernel driver is precisely what keeps your single-image OS architecture clean — the reason to pick AMD over NVIDIA here is architectural, not benchmark-driven.	USE
open-webui/open-webui	BSD-3	A complete chat front end you can point at your stack today. Useful as internal tooling and as a design reference; not the ten-foot UI you will ship.	STUDY

LAYER 05

Voice — speak to it and it works

Do not assemble this from scratch. The Wyoming protocol already defines the whole pipeline — wake word, speech-to-text, intent, speech-out — as swappable services, and it has been running in millions of homes for years.

PROJECT	LICENSE	ROLE	VERDICT
rhasspy/wyoming	MIT	The glue. A simple protocol connecting each voice stage as an independent service. Adopt it and every component below becomes swappable without rewriting your pipeline.	USE
dscripka/openWakeWord	Apache-2.0	Wake word detection. Train your own brand word — the phrase people say to your box is part of the product, not a config value.	USE
snakers4/silero-vad	MIT	Voice activity detection — knowing when the speaker stopped. Small, fast, and the difference between a natural exchange and an awkward one.	USE
SYSTRAN/faster-whisper	MIT	Speech to text. Sub-second on real hardware. Pair with whisper.cpp if you want one inference runtime across the whole box.	USE
hexgrad/kokoro	Apache-2.0	Text to speech, 82M parameters. Natural output at a size that runs alongside everything else. This is your voice.	USE

PROJECT	LICENSE	ROLE	VERDICT
rhasspy/piper	MIT	Archived October 2025 and read-only. Still works, still recommended in old tutorials — do not start a new product on it. Go to Kokoro.	SKIP
pipecat-ai/pipecat	BSD-2	Real-time conversational voice with interruption handling and barge-in. Reach for it when the wall box needs to hold a phone conversation rather than answer a command.	STUDY
speaches-ai/speaches	MIT	Wraps local speech-to-text and text-to-speech behind an OpenAI-compatible API. Convenient seam if you ever want to swap a cloud voice in for a customer who asks.	USE

LAYER 06

Orchestration, tools and automations

PROJECT	LICENSE	ROLE	VERDICT
modelcontextprotocol	MIT	How agents call tools. Spec plus SDKs plus a library of existing servers. Every capability on your box becomes an MCP server, and you inherit an ecosystem instead of inventing a plugin format.	USE
node-red/node-red	Apache-2.0	Visual flow automation with a huge node library, genuinely open licensed, and light enough to run on the appliance. The right engine under your trigger-and-action layer.	USE
n8n-io/n8n	Sustainable Use	Excellent product, but not open source — its license restricts commercial hosting and redistribution. Shipping it inside a product you sell needs a commercial agreement. Know this before you build on it.	CAREFUL

PROJECT	LICENSE	ROLE	VERDICT
temporalio/temporal	MIT	Durable execution — a flow survives a crash, a reboot, or a phone that fell asleep mid-task, and resumes exactly where it stopped. Heavy, but retries and resumability are not optional when real invoices are involved.	CAREFUL
langchain-ai/langgraph	MIT	Stateful agent graphs with checkpointing and human-in-the-loop interrupts — which maps almost directly onto your approval board.	STUDY

LAYER 07

Interface, apps and data

PROJECT	LICENSE	ROLE	VERDICT
flutter/flutter	BSD-3	One codebase for the wall screen and the phone app. Permissive license with no commercial-embedded complications, unlike the Qt/QML route Omarchy took — which matters the moment you sell hardware.	USE
cage-kiosk/cage	MIT	A Wayland compositor that runs exactly one application fullscreen. Your ten-foot UI in kiosk mode, with no desktop for anyone to fall out of.	USE
flatpak/flatpak	LGPL-2.1	Sandboxed apps versioned separately from the OS. Pair with xdg-desktop-portal so file, camera and mic access is granted per app by a person, not assumed.	USE
89luca89/distrobox	GPL-3.0	The escape hatch. Full mutable Linux with root in a container, quarantined from the host. This is how you offer total freedom without any risk to the appliance.	USE

PROJECT	LICENSE	ROLE	VERDICT
asg017/sqlite-vec	Apache-2.0 / MIT	Vector search inside SQLite. One file holds the business's whole memory — easy to encrypt, easy to back up, easy to hand back when they leave.	USE
restic/restic	BSD-2	Encrypted backup to a destination the customer chooses — their NAS, their bucket. They are protected from a dead drive; you never hold a key.	USE
juanfont/headscale	BSD-3	Self-hosted control plane for a WireGuard mesh, so the phone app reaches the box directly. Keeps your "no pipe into the customer's machine" posture intact.	USE

ASSEMBLY

What to clone this week

In order. Each step produces something you can see working before the next one starts.

-
- 01** **scrcpy** — get a real phone mirrored on a Linux box with working input. One afternoon.
 - 02** **uiautomator2** — dump the UI tree of the Google Business Profile app and tap a button from Python. Prototype before you write a service.
 - 03** **maestro** — read the YAML flow format and steal its element-matching and waiting logic. Don't solve that twice.
-

- 04 **llama.cpp + llama-swap** — one endpoint, two models, hot swap on demand. Your orchestrator's engine room, running in an hour.

- 05 **wyoming + openWakeWord + faster-whisper + kokoro** — wire the four together and say a sentence to your box. This is the demo people remember.

- 06 **home-assistant/operating-system** — read the update system docs end to end before you commit to your own OS architecture.

- 07 **bootc + greenboot** — build one image, boot it, break it on purpose, watch it roll itself back.

The order matters more than the list

Steps one through three are the product. Steps four and five are the demo. Steps six and seven are the company. Anyone can clone all sixty repos in this catalog in a weekend; the useful question is which three you have working by Friday.

Licenses noted where they affect shipping hardware commercially — verify each against its repository before you depend on it, since projects relicense. Two flagged above are live traps: Piper is archived, and n8n is not open source.

RESEARCH + DESIGN

The App Store Layer

The package format, the distribution model, the trust ladder, the installer, and what Grok Bot and Apple are actually shipping. Everything from the second half of the session.

01

The package format — don't invent one

Two published vendor-neutral specs already exist, with a technical charter and a reference validator. Your own ledger said "BUILD YOUR EXTENSIONS, reuse plugin standards underneath." Here is the standard.

Agent Plugins 1.0.0 — agent-plugins.org

plugin.json at the package root. Strict allowed-field list:

```
$schema  name  version  description  author
homepage repository license keywords extensions
```

- Bundles exactly two component types in v1: **Skills** (from skills/) and **MCP servers** (via mcp.json)
- Directory-based, not archives. SemVer recommended.
- Path containment — anything resolving outside the plugin root is rejected
- Subprocess isolation via PLUGIN_ROOT and PLUGIN_DATA
- \$schema declares the target spec version; a schema change requires a new spec release

THE GAP Straight from the spec: **"The specification contains no explicit capability or permission system."** That's where you go, and extensions is the designed entry point.

Agent Skills — agentskills.io

SKILL.md with YAML frontmatter: name, description (max 1024 chars), license, compatibility, metadata, allowed-tools.

Progressive disclosure, which maps exactly onto your dumb-agent design:

~100 tokens	name + description, loaded at startup for EVERY skill	→ routing
<5000 tokens	the SKILL.md body, loaded only on activation	→ instructions
as needed	scripts/ references/ assets/	→ resources

That first tier *is* your router. And the description doubles as the trigger rule — the real Neon skill reads as a list of phrases: *"object storage" or "S3-compatible storage" → ...; "database" or "Postgres" → ...*

Reference validator: `skills-ref validate ./my-skill`.

Your VAPP manifest

```
{
  "$schema": "https://agent-plugins.org/schemas/1.0.0/plugin.schema.json",
  "name": "restaurant-menu",
  "version": "1.2.0",
  "description": "QR menu with verified item ratings.",
  "license": "Apache-2.0",
  "extensions": {
    "io.anextgent": {
      "vertical": "restaurant",
      "reads": ["item.list", "review.aggregate_by_item", "business.get"],
      "writes": ["visit.record_scan"],
      "surfaces": { "public_page": {...}, "qr": {...}, "agent_api": {...} },
      "settings": { "show_ratings": {...} },
      "requires": { "ingest": ["pos"] },
      "events": { "emits": [...], "listens": [...] }
    }
  }
}
```

Five things you get free: a VAPP is a valid Agent Plugin and installs in any conformant client · skilla is your installer · the local registry.json pattern is your install table · path containment and PLUGIN_ROOT are your sandbox boundary, already specified · skills-ref is a validator you didn't write.

Caveat: both specs are young. Check governance and adoption before depending on their roadmap — though extensions means your fields don't ride on their evolution either way.

02

The distribution model — steal xAI's

Read from the actual xai-org/plugin-marketplace repo, not from marketing.

The repo is an index, not a store.

From their CONTRIBUTING: *"This repo is an index: a PR doesn't ship a plugin's source."*
They host nothing. The catalog holds pointers.

```
{
  "name": "vercel",
  "description": "Vercel deployment platform integration...",
  "category": "deployment",
  "source": {
    "source": "url",
    "url": "https://github.com/vercel/vercel-plugin.git",
    "sha": "df0f55213f7b8db23a3ee7f27511ed344cdb2c74"
  },
  "homepage": "...", "keywords": [...], "domains": ["vercel.com"]
}
```

This is the direct answer to *"I want the least control possible."* You don't hold the code. You hold the index and the signature. Zero hosting cost, zero distribution liability, the developer keeps ownership — and you still control what enters the catalog.

Four mechanics to copy exactly

- **Full 40-character commit SHA.** No branches, no tags, no abbreviated SHAs — the validator rejects all of them. Their reason: *"A moving ref would let a later force-push ship new code to everyone silently."* One line of validation, enormous value.

- **Generated index.** `generate-plugin-index.py` fetches each pinned SHA and extracts the real components. The catalog metadata is *derived* from the pinned code, so it can't drift or lie about what a package contains.
- **Local plugins need a README and a valid manifest.** Remote ones vendor nothing.
- **CI validator** — `validate-catalog.py`. That's your certification tier one, free.

Review also checks source legitimacy — official org versus a throwaway account, repo exists, brand matches source. Cheap, manual, and it stops the obvious attacks.

THEIR GAP `plugin.json` has **no permission model** — no capabilities, no scopes, no risk levels. Trust is binary: `grok plugin install <name> --trust`. Fine for a developer's laptop. Not fine for a business's operational data. That's your improvement.

03

The trust ladder — Red Hat's four stores

NAMING TRAP RHEL "AppStream" is not an app store. In RHEL 8+ the distro splits into BaseOS and AppStream *repositories*. Since `ximion/appstream` (the freedesktop metadata standard) is also on your list, you have three AppStreams in play. Rename yours before it costs a week.

THING	LAYER	WHAT TO TAKE
Ecosystem Catalog	index	Index-not-store, certification badges visible in the listing, hardware certification as its own track
Red Hat Marketplace	commerce	Entitlement and metering bound to the catalog entry — Candlepin + Lago in your stack
Automation Hub	content	Your actual model. See below.
Developer Hub (Backstage)	developer portal	Software catalog, templates, docs. Later.

Why Automation Hub, not the container catalog

Ansible Collections are packaged, versioned capability content — modules, roles, plugins, playbooks bundled as one installable unit. That's structurally a VAPP, not an application. And the three-tier structure is exactly what you need:

GALAXY	free, public, community anyone with a GitHub account publishes no support claim, no guarantee
AUTOMATION HUB	curated, certified, supported subscription-gated explicit support lifecycle per collection
PRIVATE AUTOMATION HUB	the org runs their own signs and publishes their own content air-gap capable
clients check Hub first, fall back to Galaxy	

RED HAT	YOURS
Galaxy	community — anyone publishes, no guarantee
Automation Hub certified	verified / certified — you tested it, you support it
Private Automation Hub	the per-business catalog on their node

Certification isn't overhead. It's the revenue line.

The catalog is free discovery; certification and the support commitment are what people pay for. That's the Red Hat innovation and it's the one you're copying at the business level.

What Red Hat gives you: the backend — catalog, certification, signing, entitlement, lifecycle, tiering. The boring, hard-to-design half, already solved.

What it doesn't: the frontend. Their stores serve enterprise IT buyers evaluating vendor software over weeks. There is no Red Hat equivalent of a per-business grant screen. Nobody in that lineage has ever had to make install feel like nothing.

04

The installer — you already listed it

junior/skilla — item #107 on your build list. MIT, pure bash, needs only git and jq. It already does the whole job:

- Clones a repo, discovers `skills/<name>/SKILL.md`
- Resolves declared `requires:` dependencies
- Installs to `.agents/skills/` (project) or `~/.agents/skills/` (global)
- **Tracks installs in a small `registry.json`** so list, update and remove work cleanly
- For Agent Plugins 1.0.0 packages: validates the manifest, installs the whole package, expands `${PLUGIN_ROOT}` and `${PLUGIN_DATA}` in MCP config

DON'T BUILD An installer. That `registry.json` pattern is your `install` table on the node — proven, tiny, and already written.

05

Three targets per app

Apple ships one artifact: a binary that runs on a device. Yours ships up to three deployment targets in one package, and the store has to understand all of them.

NODE half	runs in WASM on the business's node ingestion parsers · capability providers · automations scheduled jobs · event handlers sees private data, never leaves the node
PUBLIC half	rendered by the edge from declared blocks page · widget · QR target · JSON-LD · MCP ZERO auth, fully cacheable, consumed by people who are not your customers and will never log in
OWNER half	wall screen tiles and phone app views also declared blocks, rendered by the platform

It maps cleanly onto what Agent Plugins already gives you:

skills/	→ what the agent can do	(node half)
mcp.json	→ tools it exposes	(node half)
extensions.io.anextgent		
└ reads / writes	→ capability wiring	(node half)
└ surfaces.public_page	→ blocks	(public half)
└ surfaces.qr / widget	→ blocks	(public half)
└ settings	→ owner UI schema	(owner half)

Your store is also **three-sided** where Apple's is two: developer → business → that business's customers. Certification checks all three halves; the grant screen covers only the node half, because the public half touches nothing private by construction.

06

Nine deployable units

Contracts make things go together. Deployments make them separate. One repo everything depends on, that depends on nothing.

1	contracts	schemas, capability defs, manifest, ActionSpec
2	app-store	Agent Plugins registry: catalog, SHA pinning, generated index, validator, CI
3	flow-engine	map once → replay → fingerprint → verify → push
4	ingest	email → structured business data, versioned parsers
5	record + resolver	canonical truth with provenance and conflict policy
6	edge	record → public page, widget, JSON-LD, MCP
7	control-plane	capability registry, policy, constitution, ledger
8	voice	Wyoming pipeline wired to a structured record
9	os	bootc image, kiosk shell, app store, recovery

Nine projects that compose — the way Foreman, Katello, Pulp and Candlepin compose. Each one is something someone else would run without the rest.

Five rules that make it actually modular

1. **No shared database between components.** If two read the same tables they're one component wearing two names. This is the rule that breaks first and costs most.
2. **Every dependency crosses a contract.** A never imports B — it calls a declared interface with a fake available for testing.
3. **Each ships its own container and starts alone.** If it crashes without another running, the boundary is wrong.
4. **Each has its own version and cadence.** contracts is semver'd; everything declares which version it implements.
5. **Each has a README that doesn't mention the others.** That's the real test.

WARNING Modular **boundaries** from day one. Modular **deployment** only when you need it. Nine services with a team of one means you spend the year on plumbing. Define the boundaries, run them in one process, split later when you've felt the pain.

The App Store is the right one to build separately first — zero dependency on the record, the node, or any app existing. It forces the manifest to be right before anything is built on it, and it's the piece with the clearest open-source story.

Grok Bot

Launched 11 August 2026, early beta. Template marketplace 28 August. Grok Build plugin marketplace June 2026, 220 plugins.

What it is

"An agent with a computer" — a **cloud-based desktop** with its own filesystem, terminal and apps. Role-based bots with their own logins, always-on. Uses apps, browses the web, writes and runs code. Taught by prompts **or screen recordings**. Bots trigger each other. Routines and event triggers. MCP servers, plugins, skills — Gmail, Calendar, Drive, multiple accounts per service.

Control: natural-language permission rules, a **separate review agent that approves or blocks**, allow/block lists, explicit authorization for significant actions, escalation to humans.

Access: requires a \$120–200/mo Cursor or SuperGrok Heavy subscription. Cursor Pro lists it at \$20/mo, Pro+ \$60, Ultra \$200, with unpublished weekly allowances. **Grok Bot is not open source** — Grok Build (the coding agent and TUI) is, and the marketplace index is.

The overlap is real

- Agent with its own computer and its own logins
- **Taught by screen recording** — that's record-and-replay
- **Natural-language permission rules** — that's a Constitution
- **A separate reviewer that approves** — actor isn't approver, same principle as verifier ≠ executor
- Human escalation, a template marketplace, routines and event triggers

Seven ways it is structurally not your product

1. **It's a cloud computer.** The thing you're positioned against, and they can't flip it — cloud is their business model.
2. **The buyer is a knowledge worker.** Personal CRM, fitness coaching, code generation, searching Slack and Notion. Not a charter captain.
3. **The plugins are enterprise SaaS.** Workspace, Slack, M365, Salesforce, Jira. Not Toast, Square, FareHarbor, Vagaro, Clover — the exact set small businesses run.
4. **No physical device, no real SIM.** A cloud desktop cannot operate an Android app needing a logged-in mobile session with SMS 2FA on the business's own number.

5. **No verification layer.** The reviewer approves *before*. Nothing independently confirms the world changed *after*.
6. **No business data plane.** No canonical truth, no verified reviews, no availability aggregation, no consented grant model.
7. **No payment rail.**

LIABILITY Their terms put account risk on the customer. A business banned from Toast for automated access on a cloud VM is the business's problem. Yours: their device, their logins, human speed, a person approving each action, and a screenshot of what happened. Same activity, very different risk profile — and that's a sales answer, not just a legal one.

08

Apple

September 9, 2026 event. Foldable iPhone (Duo/Ultra) at \$1,999, iPhone 18/18 Pro, AirPods 5, Watch Series 12 — with John Ternus debuting as CEO.

They are building a version of this, and it's gated

- **Siri rebuilt** — LLM-based, multi-step requests, running on **Google Gemini models underneath**
- **App Intents is the mechanism.** WWDC 2026 declared it "the central building block" of agentic iOS. Siri reaches into an app, retrieves information and invokes actions *without opening it*. Shipping with the fall OS releases.
- **Apple Business** — Business Connect, Business Manager and Business Essentials merged into one platform in April 2026. Siri and Maps pull recommendations directly from it, across a billion devices.
- **Maps ads launched summer 2026** in the US and Canada, sold through the same platform.

The gate, precisely

App Intents = a capability registry. Apple Business = a business record. Agentic Siri = an orchestrator. Same architecture. But both sides require **voluntary adoption on Apple's terms**: the developer must implement App Intents, and the business must claim and maintain a listing.

Apple cannot operate an app that didn't cooperate. You can. That's not a gap they can close — it's a consequence of being the platform.

58% of U.S. businesses haven't claimed their Apple listing. Only 16% actively manage it.

Apple has a billion devices, free tooling, Siri distribution and an ads business riding on it — and 84% of businesses don't maintain the data. Not because they don't care. Because maintaining it is unpaid work.

That is exactly what your text-back toggle solves. **Apple has the distribution and no mechanism. You have the mechanism.**

Three pieces of ammunition

- **Maps ads are real and shipped this summer.** Apple now monetizes the listing — renting a business access to its own presence. Fact, not rhetoric. It belongs in the 1984 remake.
- **Siri runs on Gemini.** A \$3-trillion company rented someone else's model. Models are commodity; the substrate isn't.
- **Apple just made local business data strategically important.** Every small business is about to hear they need to manage their Apple listing. You arrive with the thing that does it automatically.

MOVE **Feed Apple, don't fight it.** Apple Business becomes one more syndication destination. Your businesses rank in Siri while competitors' listings sit unclaimed — and the sales line is *"Siri is answering questions about you right now with bad data. I fix that too."*

Where they genuinely threaten you: the consumer discovery query. If Siri answers "who has crab legs nearby" from Apple Business data across a billion devices, that's your query on their distribution. It only works when the data is current, and 84% of it isn't.

The export and the bookkeeper

The box structures their private data into something they own — a spreadsheet, regenerated every morning, sitting on their machine.

- **It makes "you own it" legible.** A charter captain doesn't understand a consented federated data plane. He understands a file with all his stuff in it.
- **It's the shutdown test answered with an object.** "If I go out of business your machine keeps working" is a claim. A spreadsheet updated this morning is proof.
- **It kills API dependency for reads permanently.** No rate limits, no partner program, no access to lose.
- **Spreadsheets are where every small business already lives** — their bookkeeper, accountant, lender and insurance agent all work in Excel.
- **Portability increases retention.** Products that make leaving easy keep people.

Four design calls

1. **The spreadsheet is an export, not the source.** Canonical stays in Postgres on the node; the workbook is regenerated from it. Otherwise you inherit corruption and no referential integrity.
2. **Local file by default, not Google Sheets.** Sheets sends the data to Google, which contradicts everything else. Optional sync to a destination they pick.
3. **One workbook per domain, stable schema.** Never reorder or rename columns — their accountant has formulas pointed at those cells.
4. **The agent queries the database, not the spreadsheet.** Humans read the workbook.

Bookkeepers are an unworked channel

Each one serves 30–80 small businesses. Every month they chase numbers scattered across four systems — it's literally their billable time being wasted. Hand them a client whose numbers arrive clean and on time and they will ask what it is.

One bookkeeper who likes it is fifty warm introductions, from someone the owner already trusts with their money. Same shape as the chamber play, better qualified, and nobody is competing for that relationship.

The line: *"Every month your bookkeeper asks for numbers you have to dig out of four different systems. This just has them. Updated every morning, in a spreadsheet, on your computer."*

Research current as of September 2026. Product names, pricing and feature claims should be re-verified against first-party sources before use in legal, investment or public marketing contexts. Companions: the Playbook, the Build Spec, the Build Plan, the Parts Catalog, An Honest Read.